

Multiagentní systémy

Učební text ke státní závěrečné zkoušce

SZZ Umělá inteligence, MFF UK

srpen 2026

Text pokrývá okruh „Multiagentní systémy“ magisterské SZZ oboru Umělá inteligence. Kapitoly odpovídají **jedna ku jedné** větám oficiálního znění okruhu (viz mapovací tabulka níže). Vychází ze slidů přednášky NAIL106 *Multiagentní systémy* (Neruda, Pilát, MFF UK) a z učebnic M. Wooldridge: *An Introduction to MultiAgent Systems* (2. vyd., 2009), Y. Shoham, K. Leyton-Brown: *Multiagent Systems* (2009) a S. Russell, P. Norvig: *Artificial Intelligence: A Modern Approach*. Anglické termíny jsou při prvním výskytu uvedeny kurzívou v závorce, protože zkoušející je běžně používá a literatura je výhradně anglická.

Rozsah textu a značení

Rozsah kapitol odpovídá váze látky: nejvíce prostoru dostávají architektury agentů, komunikace a kooperace s teorií her, tedy to, čemu se přednáška věnuje nejpodrobněji.

Značkou [♦ nad rámec slidů] je označena látka, která **není ve slidech** přednášky, ale je v textu ponechána — buď proto, že ji *jmenuje oficiální znění okruhu*, nebo proto, že bez ní nedává okolní výklad smysl. Slouží jako nabídka ke škrtnutí: co označeno není, je ve slidech doloženo.

Celé kapitoly 3 (hledání cesty) a 6 (učení agentů) ve slidech nejsou — přednáška je explicitně uvádí mezi nepokrytými tématy — ale okruh je vyjmenovává, takže je nutné je umět. Jsou proto zpracovány stručně, na úrovni, která na zkoušce obstojí.

Mapování okruhu na kapitoly

Věta oficiálního znění okruhu	Kapitola v tomto textu
Architektura autonomního agenta; mechanismus výběru akcí.	1 Architektura autonomního agenta a mechanismus výběru akcí
Metody pro řízení agentů; symbolické a konekcionistické reaktivní plánování, hybridní přístupy.	2 Metody pro řízení agentů
Problém hledání cesty.	3 Problém hledání cesty
Komunikace a znalosti v multiagentních systémech, ontologie, řečové akty, FIPA-ACL, protokoly.	4 Komunikace a znalosti v MAS
Distribuované řešení problémů, kooperace, Nashova rovnováha, Paretova efektivita, alokace zdrojů, aukce.	5 Distribuované řešení problémů, kooperace, teorie her a aukce
Metody pro učení agentů; zpětnovazební učení.	6 Učení agentů a zpětnovazební učení
Metodologie návrhu, jazyky a prostředí multiagentních systémů.	7 Metodologie návrhu, jazyky a prostředí MAS

Obsah

Mapování okruhu na kapitoly	1
1 Architektura autonomního agenta a mechanismus výběru akcí	4
1.1 Agent a multiagentní systém	4
1.2 Vlastnosti inteligentního agenta	5
1.3 Intencionální systémy a intencionální postoj	5
1.4 Prostředí a jeho vlastnosti	5
1.5 Abstraktní architektura agenta	6

1.6	Čistě reaktivní agent a agent s vnitřním stavem	7
1.7	Utilita: jak agentovi říct, co má dělat	7
1.8	Predikátová specifikace úlohy	8
1.9	Mechanismus výběru akcí — přehled	9
1.10	Shrnutí ke zkoušce	9
2	Metody pro řízení agentů: symbolické a konekcionistické reaktivní plánování, hybridní přístupy	9
2.1	Klasický přístup UI a deduktivní agent	10
2.2	Praktické uvažování: architektura BDI	11
2.3	Plánování: STRIPS	12
2.4	Implementace BDI agenta	13
2.5	Odhodlnání agenta (<i>commitment</i>)	14
2.6	Procedurální odvozovací systém (PRS)	14
2.7	Reaktivní přístup a jeho východiska	15
2.8	Symbolické reaktivní plánování: subsumpční architektura	16
2.9	Konekcionistické reaktivní plánování: agent network architecture	17
2.10	Omezení reaktivních architektur	18
2.11	Hybridní architektury	18
2.12	Komplexní architektury: IDA a globální pracovní prostor	19
2.13	Srovnání architektur	20
2.14	Shrnutí ke zkoušce	20
3	Problém hledání cesty	21
3.1	Formulace a zařazení	21
3.2	A*	22
3.3	Dynamické prostředí: D* Lite	23
3.4	Hledání cest pro více agentů (MAPF)	23
3.5	Shrnutí ke zkoušce	24
4	Komunikace a znalosti v MAS: ontologie, řečové akty, FIPA-ACL, protokoly	24
4.1	Od agenta k multiagentnímu systému	24
4.2	Co je ontologie	25
4.3	Ontologické jazyky	25
4.4	Uvažování o znalostech: epistemická logika	27
4.5	Proč je komunikace agentů jiná	27
4.6	Teorie řečových aktů	27
4.7	KQML a FIPA-ACL	28
4.8	Interakční protokoly	30
4.9	Shrnutí ke zkoušce	30
5	Distribučované řešení problémů, kooperace, teorie her a aukce	31
5.1	Koherence a koordinace	31
5.2	Sdílení úloh a sdílení výsledků	31
5.3	Contract Net	32
5.4	Systémy s tabulí (blackboard)	32
5.5	Distribučované omezující podmínky (DisCSP a DCOP)	33
5.6	Interakce agentů v prostředí	33
5.7	Sobecký agent	33
5.8	Hra v normální formě	34
5.9	Klasické hry	35
5.10	Sociální volba a volby	36
5.11	Aukce jako mechanismus alokace zdrojů	37

5.12	Čtyři základní typy aukcí	38
5.13	Kombinatorické aukce a VCG	39
5.14	Další typy aukcí a alokace zdrojů	39
5.15	Návrh mechanismů, VCG a vyjednávání	39
5.16	Shrnutí ke zkoušce	40
6	Metody pro učení agentů a zpětnovazební učení	41
6.1	Proč a jak se agenti učí	41
6.2	Markovský rozhodovací proces	42
6.3	Q-learning a SARSA	43
6.4	Metody založené na politice	44
6.5	Multiagentní zpětnovazební učení	44
6.6	Shrnutí ke zkoušce	45
7	Metodologie návrhu, jazyky a prostředí MAS	46
7.1	Agentově orientované softwarové inženýrství	46
7.2	Standard FIPA a architektura platformy	46
7.3	Jazyky a prostředí	46
7.4	Agenti postavení na velkých jazykových modelech	47
7.5	Shrnutí ke zkoušce	48
	Závěrem: jak na zkoušku	48

1 Architektura autonomního agenta a mechanismus výběru akcí

1.1 Agent a multiagentní systém

Multiagentní systémy (*multiagent systems*, MAS) vznikly jako samostatná oblast informatiky v 90. letech jako odpověď na souběžné trendy: ubikvitnost výpočetní techniky, konektivitu a distribuovanost (výpočet jako interakce), rostoucí složitost úloh, delegaci úloh na software a tlak na snadnost použití. Klasické paradigma „program dostane vstup, spočítá výstup, skončí“ přestává stačit, jakmile je software trvale zasazen do prostředí, které se mění nezávisle na něm.

Agent — pracovní definice

Russell, Norvig (AIMA): Agent je cokoli, co lze chápat jako entitu *vnímající* své prostředí prostřednictvím senzorů a *působící* na toto prostředí pomocí efektorů.

Pattie Maes: Autonomní agenti jsou výpočetní systémy, které obývají nějaké složité dynamické prostředí, autonomně v něm vnímají a jednají, a tím naplňují množinu cílů či úkolů, pro které byly navrženy.

Wooldridge, Jennings: Agent je počítačový systém zasazený do nějakého prostředí, schopný v tomto prostředí *autonomního jednání* vedoucího ke splnění delegovaných cílů.

Jednotná definice neexistuje — Franklin a Graesser (1996) v článku *Is it an agent, or just a program?* shromáždili desítky vzájemně nekompatibilních definic. Klíčové je odlišit agenta od běžné procedury či objektu.

Multiagentní systém

Multiagentní systém je systém složený z více agentů, kteří spolu *interagují* — nejčastěji výměnou zpráv po počítačové síti. Agenti *nemusí sdílet společný cíl*; mohou mít vlastní, i protichůdné zájmy. Proto je nutné studovat mechanismy vyjednávání, koordinace a dynamického rozdělování zdrojů.

Autonomie. Člověk je plně autonomní, metoda v Javě vůbec. Agent leží mezi: měl by autonomně volit *způsob*, jak zadaný problém vyřešit (tedy volbu podcílů), nikoli však hlavní cíl — ten je mu delegován.

Vymezení vůči příbuzným oborům. Šíří okruhu vysvětluje to, že MAS lze číst z pěti různých stran:

- **Softwarové inženýrství.** Agent je dalším stupněm abstrakce v řadě strojový kód → strukturované programy → objekty → distribuované objekty (CORBA) → agenti. Klasický bonmot: „objekty to dělají zadarmo, agenti proto, že chtějí“.
- **Distribuované a paralelní systémy.** Desítky let výzkumu vyřešily mutex, deadlock či konsensus. Rozdíl je v tom, že agenti jsou *autonomní*, koordinační mechanismy nejsou dané dopředu a vše se musí vyřešit až za běhu.
- **Umělá inteligence.** Tradičně: MAS je podoblast UI. Podle Russella a Norviga je naopak cílem UI návrh inteligentních agentů (tedy $UI \subseteq MAS$); Etzioniho vyhrčené „MAS je 1 % UI a 99 % informatiky“. MAS používá techniky UI (reprezentace znalostí, plánování), ale zdůrazňuje *sociální* aspekty přehlížené klasickou UI.
- **Teorie her a ekonomie.** Formální aparát pro racionální rozhodování více subjektů. Matematická teorie her řeší hlavně otázky *existence*, MAS naopak *výpočetní* aspekty (složitost, praktická použitelnost).
- **Sociologie.** Klíčovým pojmem MAS jsou *společnosti agentů*. Platí zde stejná výhrada jako u vztahu UI a lidské inteligence: inspirace ano, nutnost ne (letadla nemávají křídly). Opačným směrem je vazba pevnější: sociologie, ekonomie i ekologie používají agenty jako stavební kámen svých simulačních modelů — *agentové modelování* (*agent-based modelling*, ABM).

1.2 Vlastnosti inteligentního agenta

- **Reaktivita** (*reactive*) — vnímá prostředí a je schopen reagovat na jeho změny v rozumném (reálném) čase.
- **Proaktivita** (*proactive*) — má vlastní cíle a aktivně je naplňuje, z vlastní iniciativy, ne jen jako odpověď na podnět.
- **Sociálnost** (*social ability*) — umí komunikovat s jinými agenty a s lidmi.

Dosáhnout *čistě* reaktivního nebo *čistě* cílově orientovaného chování je poměrně snadné. Obtížné je najít **rovnováhu mezi reaktivitou a proaktivitou**: agent nesmí slepě následovat plán, který se stal nesmyslným, ale ani nesmí neustále přehodnocovat cíle a nikam nedojít. Tento kompromis se vrací v celém textu — viz zejména odhodlání agenta (odst. 2.5) a hybridní architektury (odst. 2.11).

1.3 Intencionální systémy a intencionální postoj

Když o agentech mluvíme, přisuzujeme jim *mentální stavy*: „agent věří, že...“, „agent chce...“. Daniel Dennett definuje **intencionální systém** jako entitu, jejíž chování lze predikovat přisouzením vlastností jako přesvědčení (*beliefs*), touhy (*desires*) a racionalita. Intencionální systémy tvoří hierarchii: systém *prvního řádu* má přesvědčení a touhy, systém *druhého řádu* má přesvědčení a touhy o přesvědčeních a touhách (svých i cizích), atd.

Dennett rozlišuje tři *postoje* (*stances*) k predikci chování systému: *fyzikální* (stačí fyzikální zákony — hodím-li kamenem, nemusím mluvit o jeho touhách, stačí hmotnost, rychlost a gravitace), *návrhový* (*design stance*: neznám všechny fyzikální zákony, ale vím, k jakému účelu byl systém navržen — umím nastavit budík, aniž bych rozuměl jeho elektronice) a *intencionální* (popisují systém pomocí přesvědčení, tužeb a racionality).

Shoham uvádí provokativní příklad: „Vypínač v místnosti je velmi kooperativní agent, který je z vlastní vůle ochoten vést elektrický proud, ale činí tak jen tehdy, věří-li, že si to skutečně přejeme.“ Popis je konzistentní a elegantní, přesto většině lidí připadá absurdní — *protože pro vypínač máme jednodušší a stejně přesný popis*. Intencionální postoj se vyplatí tam, kde fyzikální či návrhový popis neexistuje, nebo je nepraktický (i se schématem počítače je těžké odvodit, proč se po kliknutí na ikonu otevře okno). **Z pohledu informatiky je intencionální postoj především abstrakce pro zvládnutí složitosti** — a právě to je pro mnoho programátorů hlavní přínos MAS. Dennett sám dodává filozofickou otázku: intencionalita podle něj neexistuje sama o sobě, vzniká teprve *v procesu interpretace* pozorovatelem.

1.4 Prostředí a jeho vlastnosti

Chování agenta zásadně určuje typ prostředí, ve kterém se nachází. Standardní klasifikace (Russell, Norvig):

Dichotomie	Význam
plně / částečně pozorovatelné (<i>fully / partially observable</i>)	Senzory agenta mu dávají úplný stav prostředí — nebo jen jeho část (skryté informace, šum, omezený dosah).
statické / dynamické (<i>static / dynamic</i>)	Prostředí se mění pouze v důsledku akcí agenta — nebo i jinak (jiní agenti, přírodní děje). Ve statickém prostředí si agent může „v klidu rozmyslet“ další tah.
deterministické / nedeterministické	Každá akce má právě jeden možný výsledek — nebo více (se známým či neznámým rozdělením pravděpodobnosti; pak mluvíme o stochastickém prostředí).
diskrétní / spojitý	Konečný počet stavů a akcí — nebo spojitý veličiny (poloha, rychlost).

Nepříjemné konstatování: většina zajímavých prostředí (reálný svět, internet) je spojitá, nedeterministická, dynamická a jen částečně pozorovatelná. Právě proto nestačí naprogramovat agenta jako pevnou tabulku pravidel.

Příklad — termostat. Ne příliš inteligentní agent, zasazený do prostředí (místnost). Dva vjemy („je zima“, „teplota v pořádku“), dvě akce (zapnout, vypnout), rozhodovací jednotka: zima $\Rightarrow$ zapni, OK $\Rightarrow$ vypni. Prostor je přesto nedeterministické (někdo otevře dveře).

Příklad — softwarový démon. Např. unixový `xbiff`: prostředím je operační systém, vnímá voláním systémových služeb, akcemi jsou softwarové operace (spustit poštovního klienta, změnit ikonu). Rozhodovací algoritmus je na úrovni termostatu.

1.5 Abstraktní architektura agenta

Nyní formalizujeme pohled na agenta a jeho interakci s prostředím. Předpokládáme, že prostředí je diskrétní — což není omezující, každé spojitě prostředí lze diskretizovat s libovolnou přesností.

Prostředí, agent, běh

Prostředí může být v jednom z konečně mnoha diskrétních stavů, $E = \{e, e', \dots\}$. **Agent** má k dispozici konečnou množinu akcí $A = \{a, a', \dots\}$.

Běh (*run*) agenta v prostředí je posloupnost střídajících se stavů prostředí a akcí:

$$r = e_0 \xrightarrow{a_0} e_1 \xrightarrow{a_1} e_2 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} e_n.$$

Označme R množinu všech konečných běhů (nad E a A), $R^A \subseteq R$ ty, které končí akcí, a $R^E \subseteq R$ ty, které končí stavem prostředí.

Přechodová funkce prostředí (*state transformer function*)

$$\tau : R^A \rightarrow 2^E$$

Funkce τ zobrazuje běh končící akcí na množinu stavů, do nichž se prostředí může po této akci dostat. Z této definice plynou dva důsledky:

- **Prostředí má historii** — následující stav nezávisí jen na poslední akci, ale na celé historii běhu.
- **Prostředí je nedeterministické** — výsledkem je množina stavů, dopředu nevíme, který nastane.

Je-li $\tau(r) = \emptyset$, běh skončil. Dále předpokládáme, že všechny běhy skončí.

Prostředí je trojice $Env = \langle E, e_0, \tau \rangle$, kde e_0 je počáteční stav.

Agent a systém

Agent je funkce zobrazující běhy končící stavem prostředí na akce:

$$Ag : R^E \rightarrow A.$$

Agent tedy rozhoduje na základě *celé historie* systému a rozhoduje se *deterministicky*. Označme $\mathcal{A}\mathcal{G}$ množinu všech agentů.

Systém je dvojice $\langle Ag, Env \rangle$. Posloupnost $(e_0, a_0, e_1, a_1, \dots)$ je *běh agenta Ag v prostředí Env*, právě když

1. e_0 je počáteční stav Env ,
2. $a_0 = Ag(e_0)$,
3. pro každé $u > 0$ platí $e_u \in \tau((e_0, a_0, \dots, a_{u-1}))$ a $a_u = Ag((e_0, a_0, \dots, e_u))$.

Množinu všech běhů agenta Ag v Env značíme $R(Ag, Env)$.

Funkční ekvivalence agentů

Agenti Ag_1 a Ag_2 jsou **funkčně ekvivalentní vzhledem k** Env , právě když $R(Ag_1, Env) = R(Ag_2, Env)$.

Jsou **prostě funkčně ekvivalentní** (*simply functionally equivalent*), pokud jsou funkčně ekvivalentní vzhledem ke *všem* prostředím.

Funkční ekvivalence je nástroj pro srovnávání *vyjadřovací síly* architektur: říká, že vnitřní implementace je nepodstatná, pokud vede ke stejným pozorovatelným běhům.

1.6 Čistě reaktivní agent a agent s vnitřním stavem

Čistě reaktivní agent (*purely reactive / tropistic agent*)

$$Ag : E \rightarrow A$$

Reaguje pouze na aktuální stav prostředí, historii ignoruje.

Vztah k obecnému agentovi: ke každému čistě reaktivnímu agentovi existuje funkčně ekvivalentní standardní agent; opačně to *neplatí*.

Termostat: $Ag(e) = \text{off}$ pro $e = \text{„teplota OK“}$, jinak $Ag(e) = \text{on}$.

Agent s vnitřním stavem

Nechť I je množina vnitřních stavů agenta a Per množina jeho vjemů. Agent je určen trojicí funkcí:

$see : E \rightarrow Per$	(vnímání)
$next : I \times Per \rightarrow I$	(aktualizace vnitřního stavu)
$action : I \rightarrow A$	(výběr akce)

Cyklus: agent startuje ve stavu i_0 ; vnímá prostředí ($see(e)$); aktualizuje stav ($i \leftarrow next(i, see(e))$); vybere akci ($a \leftarrow action(i)$); vykoná ji a opakuje.

Věta (bez důkazu): Ke každému agentovi s vnitřním stavem existuje funkčně ekvivalentní standardní agent. Vnitřní stav tedy *nezvyšuje vyjadřovací sílu*, ale je zásadní pro *efektivitu* a *přehlednost* implementace.

Funkce see současně formalizuje **pozorovatelnost**: pokud see zobrazí dva různé stavy $e_1 \neq e_2$ na tentýž vjem, agent je nedokáže rozlišit — prostředí je pro něj jen částečně pozorovatelné. Krajní případy: $|Per| = |E|$ a see prostá (plně pozorovatelné) versus $|Per| = 1$ (agent nevidí nic).

1.7 Utilita: jak agentovi říct, co má dělat

Nechceme agenty s natvrdo zadrátovaným chováním. Chceme agentovi říct, *co* má dělat, nikoli *jak*. Standardní postup UI je definovat cíl *nepřímo*, mírou úspěšnosti.

Utilita

Utilita stavu: $u : E \rightarrow \mathbb{R}$. Cílem agenta je dostat se do stavů s vysokou hodnotou utility. Hodnocení jednotlivých stavů je krátkozraké. Proto zavádíme **utilitu běhu**:

$$u : R \rightarrow \mathbb{R}.$$

Ta lépe odpovídá agentům, kteří běží dlouhodobě. Utilitu běhu lze odvodit ze stavů např. jako utilitu *nejhoršího* navštíveného stavu nebo jako *průměrnou* utilitu navštívených stavů — co je lepší, závisí na úloze.

Optimální agent

Nechť $P(r \mid Ag, Env)$ je pravděpodobnost běhu r agenta Ag v prostředí Env , přičemž $\sum_{r \in R(Ag, Env)} P(r \mid Ag, Env) = 1$. Optimální agent maximalizuje *očekávanou utilitu*:

$$Ag_{opt} = \arg \max_{Ag \in \mathcal{AG}} \sum_{r \in R(Ag, Env)} u(r) P(r \mid Ag, Env).$$

Definice je elegantní, ale **nekonstruktivní**: neříká, jak takového agenta navrhnout. Navíc bývá obtížné utilitní funkci vůbec zadat. (Poznámka: peníze nejsou pro člověka dobrou utilitou — utilita peněz je nelineární a liší se pro bohaté a chudé.)

Příklad: Tileworld. Klasické testovací prostředí pro agentní architektury. Agenti se pohybují po šachovnici do čtyř směrů, na desce jsou díry, překážky a bloky; cílem je zatlačit bloky do děr. Prostředí je *dynamické* — díry, překážky i bloky náhodně vznikají a zanikají. Utilita běhu:

$$u(r) = \frac{N_f}{N_a},$$

kde N_f je počet děr zaplněných agentem během běhu r a N_a počet všech děr, které se během r objevily. Agent musí umět *reagovat* na změny (blok, který tlačím, zmizel) i *využívat příležitosti* (vedle mě se objevil nový blok). Tileworld se historicky používal právě ke zkoumání kompromisu mezi reaktivitou a proaktivitou (viz odst. 2.5).

1.8 Predikátová specifikace úlohy

Speciálním případem utility je zobrazení do $\{0, 1\}$: běh je buď úspěšný ($u(r) = 1$), nebo ne.

Prostředí úlohy (*task environment*)

Nechť $\Psi : R \rightarrow \{0, 1\}$ je **predikátová specifikace**: $\Psi(r)$ platí, právě když $u(r) = 1$. **Prostředí úlohy** je dvojice $\langle Env, \Psi \rangle$; specifikuje jak vlastnosti systému (přes Env), tak kritérium splnění úlohy (přes Ψ).

Označme $R_\Psi(Ag, Env) = \{r \in R(Ag, Env) \mid \Psi(r)\}$.

Kdy agent úlohu *řeší*? Existují tři výklady:

- **Pesimistický**: $R_\Psi(Ag, Env) = R(Ag, Env)$ — *všechny* běhy splňují Ψ .
- **Optimistický**: $\exists r \in R(Ag, Env) : \Psi(r)$ — *alespoň jeden* běh.
- **Realistický**: rozšíříme τ o rozdělení pravděpodobnosti a měříme úspěch pravděpodobností $P(\Psi \mid Ag, Env) = \sum_{r \in R_\Psi(Ag, Env)} P(r \mid Ag, Env)$.

Typy úloh

Úlohy dosažení (*achievement tasks*) — „něco najít“. Je dána množina cílových stavů $G \subseteq E$; $\Psi(r)$ platí, pokud se v běhu r vyskytne alespoň jeden stav z G . Typicky každá klasická úloha UI (prohledávání).

Úlohy udržení (*maintenance tasks*) — „něčemu se vyhnout“. Je dána množina „špatných“ stavů $B \subseteq E$; $\Psi(r)$ neplatí, pokud se v r vyskytne stav z B . Typicky hry, kde B jsou prohrané pozice a prostředím je soupeř.

Kombinace: dosáhnout stavu z G a přitom se vyhnout stavům z B .

1.9 Mechanismus výběru akcí — přehled

Mechanismus výběru akcí (*action selection mechanism*)

Procedura, kterou agent v každém okamžiku rozhoduje, kterou z dostupných akcí provede, na základě vjemů z prostředí a svého vnitřního stavu. **Architektura agenta** je softwarová architektura, která takový mechanismus realizuje — podle Maes (1991) „konkrétní metodologie stavby agentů, která určuje, jak lze agenta rozložit na množinu komponent a jak mají tyto komponenty interagovat, aby dohromady odpověděly na otázku, jak data ze senzorů a aktuální vnitřní stav určují akce a budoucí vnitřní stav agenta.“

Výběr akce je **klíčový problém návrhu agenta** a celá kapitola 2 není nic jiného než přehled jeho realizací:

Rodina	Mechanismus výběru akce	Typický zástupce
symbolická / deliberativní	logická dedukce nad databází formulí; plánování	deduktivní agent, STRIPS, BDI, PRS
reaktivní (symbolická)	sada pravidel „situace $\rightarrow$ akce“ s pevnou prioritou	subsumpční architektura (Brooks)
reaktivní (konekcio-nistická)	šíření aktivace v síti modulů, vyhrává nejaktivnější	agent network architecture (Maes)
hybridní	vrstvy různé abstrakce + arbitráž mezi nimi	TouringMachines, InteRRaP
koaliční	soutěž dynamicky vznikajících koalic procesů o „vědomí“	IDA / globální pracovní prostor
naučená	maximalizace naučené Q -funkce / stochastická politika	Q-learning, actor-critic (kap. 6)
generativní	stav se zakóduje do promptu, výstup jazykového modelu se čte jako akce	LLM agenti (§7.4)

1.10 Shrnutí ke zkoušce

- Agent = autonomní systém zasazený do prostředí, který v něm vnímá a jedná, aby splnil delegované cíle. MAS = systém interagujících agentů *bez nutně sdíleného cíle*.
- Tři vlastnosti inteligentního agenta: reaktivita, proaktivita, sociálnost; těžké je jejich vyvážení.
- Dennett: intencionální systém a tři postoje (fyzikální, návrhový, intencionální); intencionální postoj je abstrakce pro zvládnutí složitosti.
- Prostedí: 4 dichotomie (pozorovatelnost, statické/dynamické, determinismus, diskretnost); reálná prostředí jsou v nejtěžší variantě.
- Formalismus: E , A , běh $r = e_0 a_0 e_1 \dots e_n$, R^A/R^E , přechodová funkce $\tau : R^A \rightarrow 2^E$ (odtud historie + nedeterminismus), $Env = \langle E, e_0, \tau \rangle$, agent $Ag : R^E \rightarrow A$, systém $\langle Ag, Env \rangle$.
- Funkční ekvivalence (vzhledem k prostředí / prostá). Čistě reaktivní agent $Ag : E \rightarrow A$ (slabší); agent s vnitřním stavem (*see*, *next*, *action*) — oba převeditelné na standardního agenta.
- Utilita: $u : E \rightarrow \mathbb{R}$ vs. $u : R \rightarrow \mathbb{R}$; optimální agent maximalizuje *očekávanou* utilitu — definice je nekonstruktivní. Tileworld a $u(r) = N_f/N_a$.
- Prostedí úlohy $\langle Env, \Psi \rangle$; pesimista/optimista/realista; úlohy dosažení (G) a udržení (B).
- **Mechanismus výběru akcí** je jádro otázky: umět vyjmenovat rodiny mechanismů a jejich zástupce (tabulka výše).

2 Metody pro řízení agentů: symbolické a konekcionistické reaktivní plánování, hybridní přístupy

Tato kapitola je jádrem okruhu: probírá jednotlivé *metody řízení agenta*, tedy konkrétní realizace mechanismu výběru akcí z odst. 1.9. Postupuje od symbolického (deliberativního) pólu k reaktivnímu a končí hybridními architekturami, které oba pól spojují:

Přístup	Kde je v této kapitole
symbolické řízení (deliberativní)	deduktivní agent, BDI, STRIPS, PRS (odst. 2.1–2.6)
symbolické reaktivní plánování	subsumpční architektura (Brooks), odst. 2.8
konekcionistické reaktivní plánování	agent network architecture (Maes), odst. 2.9
hybridní přístupy	vrstvené architektury, TouringMachines, InteRRaP, IDA, odst. 2.11 a dále

2.1 Klasický přístup UI a deduktivní agent

Klasický způsob, jakým UI staví „inteligentní systém“, stojí na dvou pilířích:

1. **Symbolická reprezentace** prostředí a chování — zde konkrétně logickými formulemi.
2. **Syntaktická manipulace** s touto reprezentací — tedy logická dedukce, dokazování vět.

Teorie o chování agenta je pak zároveň programem — *spustitelnou specifikací*, která generuje konkrétní akce. Tento přístup naráží na dva klasické problémy:

- **Problém transdukce** (*transduction problem*): jak převést reálný svět do symbolické reprezentace (počítačové vidění, zpracování přirozeného jazyka, učení). Goethem řečeno: „Šedá, milý příteli, je všechna teorie, a zelený je zlatý strom života.“
- **Problém reprezentace a uvažování**: jak znalosti reprezentovat a jak nad nimi efektivně odvozovat (dokazovače, plánovače).

Odbočka: dedukce vs. indukce. *Dedukce* je odvození závěrů, které jsou pravdivé, jsou-li pravdivé předpoklady — postup od obecného ke konkrétnímu (sylogismus: „Všichni lidé jsou smrtelní, Sókratés je člověk, tedy Sókratés je smrtelný“). *Indukce* znamená, že při pravdivých předpokladech je závěr spíše pravdivý než nepravdivý — od konkrétního k obecnému. Sherlock Holmes ve skutečnosti používá indukci, i když jí říká dedukce; matematická indukce je naopak dedukce.

Deduktivní agent (agent jako dokazovač vět)

Nechť L je množina formulí predikátové logiky prvního řádu a $D = 2^L$ množina databází těchto formulí. Vnitřní stav agenta je databáze $DB \in D$. Uvažování je realizováno odvozovacími pravidly ρ :

$$DB \vdash_{\rho} \varphi \quad \text{— formulí } \varphi \text{ lze odvodit z } DB \text{ pomocí pravidel } \rho.$$

Agent je pak trojice funkcí $see : E \rightarrow Per$, $next : D \times Per \rightarrow D$, $action : D \rightarrow A$, přičemž $action$ je definována procedurou níže.

Výběr akce jako dokazování.

```

1: function ACTION( $DB : D$ ) :  $A$ 
2:   for all  $a \in A$  do
3:     if  $DB \vdash_{\rho} Do(a)$  then return  $a$ 
4:   end if
5: end for
6: for all  $a \in A$  do
7:   if  $DB \not\vdash_{\rho} \neg Do(a)$  then return  $a$ 
8: end if
9: end for
10: return null
11: end function

```

První cyklus hledá akci, kterou lze *dokázat* jako žádoucí (odvodí se $Do(a)$). Pokud žádná taková není, druhý cyklus hledá akci alespoň *konzistentní* s databází (nelze dokázat, že by se dělat neměla).

Pokud selže i to, agent nedělá nic.

Příklad: vysavačový svět 3×3. Predikáty: $In(x, y)$ (agent je na pozici (x, y)), $Dirt(x, y)$ (na (x, y) je špína), $Facing(d)$ (agent je otočen směrem d). Pravidla pro odvození akce:

$$\begin{aligned} In(x, y) \wedge Dirt(x, y) &\Rightarrow Do(suck) \\ In(0, 0) \wedge Facing(north) \wedge \neg Dirt(0, 0) &\Rightarrow Do(forward) \\ In(0, 2) \wedge Facing(north) \wedge \neg Dirt(0, 2) &\Rightarrow Do(turn) \quad \dots \end{aligned}$$

Agent tak systematicky projde svět ve tvaru „hada“ 00–01–02–12–11–10–... Všimněte si, že úklid má vyšší prioritu než pohyb (pravidlo pro *suck* je uvedeno první a ostatní pravidla mají v předpokladu negaci *Dirt*).

Klady a zápory.

- + Elegantní, s jasnou (a krásnou) sémantikou; specifikace = program.
- Prakticky nepoužitelné pro netriviální úlohy: dokazování je pomalé, nemusí skončit.
- **Problém vypočitatelné racionality** (*calculative rationality*): agent odvodí optimální akci pro stav prostředí, jaký byl *na začátku* odvozování — mezitím se prostředí změnilo a akce už optimální není. Katastrofa v rychle se měnícím prostředí.
- Těžko se navrhuje funkce *see* (jak z obrázku udělat formule, jak reprezentovat čas).

Přesto se jednotlivé prvky tohoto přístupu objevují v pozdějších architekturách, zejména v BDI.

2.2 Praktické uvažování: architektura BDI

Praktické uvažování (*practical reasoning*)

Inspirováno lidským rozhodováním. *Teoretické* uvažování (Sókratés) vede jen k tomu, co si o světě myslíme; *praktické* uvažování vede k **akcím**. Má dvě fáze:

- **Deliberace** (*deliberation*) — *čeho chceme dosáhnout* („chci dodělat magisterské studium“).
- **Uvažování o prostředcích** (*means-ends reasoning*), tedy plánování — *jak toho dosáhneme* (vytvořit plán, jak dostudovat).

A obojí nesmí trvat příliš dlouho.

Beliefs — Desires — Intentions

Beliefs (přesvědčení) — informační stav agenta, jeho znalosti o světě. V MAS jim záměrně neříkáme „znalosti“, abychom zdůraznili, že jsou *subjektivní*, *nemusí být pravdivé* a *mohou se v budoucnu změnit*. Mohou obsahovat i odvozovací pravidla umožňující dopředné řetězení.

Desires (touhy) — motivační stav agenta, cíle nebo situace, kterých by chtěl dosáhnout. Touhy *mohou být vzájemně nekonzistentní* (chci se učit i jít na pivo).

Intentions (záměry) — stavy světa, kterých se agent *zavázal* dosáhnout. Záměry vedou k akcím: agent kvůli nim plánuje. Záměry **přetrvávají**, dokud

1. jich agent nedosáhne, nebo
2. nezačne věřit, že jsou nedosažitelné, nebo
3. nepominou důvody, které k nim vedly.

Záměry navíc *omezují další deliberaci* (nebudu znovu zvažovat možnosti neslučitelné s přijatým záměrem) a *ovlivňují budoucí přesvědčení* (počítám s tím, že se mi záměr podaří).

Rozdíl mezi touhou a záměrem ilustruje Bratman (1990): „Moje touha zahrát si dnes odpoledne basketbal je pouze potenciálním vlivem na moje odpolední chování. Musí soupeřit s mými dalšími relevantními touhami... Naproti tomu jakmile *zamýšlím* hrát dnes odpoledne basketbal, je věc rozhodnuta: obvykle už nemusím dál vážit pro a proti; až odpoledne přijde, prostě svůj záměr vykonám.“

Funkce deliberace

Označme $B \subseteq Bel$ aktuální přesvědčení, $D \subseteq Des$ aktuální touhy a $I \subseteq Int$ aktuální záměry agenta. Deliberace se skládá ze tří funkcí:

- $brf : 2^{Bel} \times Per \rightarrow 2^{Bel}$ — *belief revision function*, aktualizace přesvědčení podle vjemu;
- $options : 2^{Bel} \times 2^{Int} \rightarrow 2^{Des}$ — generování možností (tužeb) z přesvědčení a stávajících záměrů;
- $filter : 2^{Bel} \times 2^{Des} \times 2^{Int} \rightarrow 2^{Int}$ — výběr záměrů z možností.

Rozdělení deliberace na *generování možností* a *filtrování* je klíčové: generování je levné a široké, filtrování je vlastní rozhodování (bere v úvahu i stávající závazky).

2.3 Plánování: STRIPS

Uvažování o prostředcích / plánování

Proces rozhodování, jak dosáhnout cíle (záměru) pomocí dostupných prostředků (akcí).

- **Vstup:** cíl (= záměr I); aktuální stav prostředí (= přesvědčení B); akce dostupné agentovi (Ac).
- **Výstup:** plán = posloupnost akcí, po jejímž provedení je cíl splněn.

STRIPS (Fikes, Nilsson, 1971) modeluje svět množinou formulí logiky prvního řádu. Každá akce je popsána *schématem* s předpoklady a efekty (přidávané a mazané fakty). Plánovací algoritmus najde rozdíl mezi cílem a aktuálním stavem a vhodnou akci ho zmenší (*means-ends analysis*). Je to hezké, ale ne příliš praktické — algoritmus se často utápí v detailech nízké úrovně.

Deskriptor akce, plánovací problém, plán

Deskriptor akce a je trojice $\langle P_a, D_a, A_a \rangle$: P_a — předpoklady (*preconditions*), D_a — mazaná množina (*delete list*), A_a — přidávaná množina (*add list*); pro jednoduchost jde o uzemněné atomické formule (bez spojek a proměnných).

Plánovací problém je trojice $\langle B_0, O, G \rangle$, kde B_0 jsou počáteční přesvědčení, $O = \{ \langle P_a, D_a, A_a \rangle : a \in Ac \}$ deskriptory všech akcí a G množina formulí popisujících cíl.

Plán $\pi = (a_1, \dots, a_n)$, $a_i \in Ac$, určuje posloupnost databází

$$B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}, \quad i = 1, \dots, n.$$

Plán je **přípustný** (*admissible*), pokud $B_{i-1} \models P_{a_i}$ pro každé i . Plán je **korektní/validní** (*correct*), pokud je přípustný a navíc $B_n \models G$.

Plánování znamená pro zadaný $\langle B_0, O, G \rangle$ najít korektní plán, nebo prohlásit, že neexistuje.

Příklad: svět kostek (*blocks world*). Predikáty: $On(x, y)$ (x je na y), $OnTable(x)$, $Clear(x)$ (na x nic není), $Holding(x)$ (rameno drží x), $ArmEmpty$. Akce:

Akce	Pre / Del	Add
$Stack(x, y)$	Pre: $\{Clear(y), Holding(x)\}$ Del: $\{Clear(y), Holding(x)\}$	$\{ArmEmpty, On(x, y)\}$
$UnStack(x, y)$	Pre: $\{On(x, y), Clear(x), ArmEmpty\}$ Del: $\{On(x, y), ArmEmpty\}$	$\{Holding(x), Clear(y)\}$
$Pickup(x)$	Pre: $\{OnTable(x), Clear(x), ArmEmpty\}$ Del: $\{OnTable(x), ArmEmpty\}$	$\{Holding(x)\}$
$PutDown(x)$	Pre: $\{Holding(x)\}$ Del: $\{Holding(x)\}$	$\{ArmEmpty, OnTable(x)\}$

Počáteční stav $B_0 = \{Clear(A), On(A, B), OnTable(B), OnTable(C), Clear(C), ArmEmpty\}$, cíl $G = \{OnTable(A), OnTable(B), OnTable(C)\}$.

Korektní plán: $\pi = (UnStack(A, B), PutDown(A))$. Ověření:

- $B_0 \models \{On(A, B), Clear(A), ArmEmpty\}$, tedy první akce je přípustná; po ní

$$B_1 = (B_0 \setminus \{On(A, B), ArmEmpty\}) \cup \{Holding(A), Clear(B)\}.$$

- $B_1 \models \{Holding(A)\}$, tedy i druhá akce je přípustná; po ní

$$B_2 = (B_1 \setminus \{Holding(A)\}) \cup \{ArmEmpty, OnTable(A)\}.$$

- $B_2 \models G$, plán je korektní.

Pomocné funkce nad plány

Plan — množina všech plánů nad *Ac*; *pre*(π) — předpoklad plánu; *body*(π) — tělo plánu (posloupnost akcí); *empty*(π) — je plán prázdný?; *execute*(π) — provede všechny akce plánu; *hd*(π) — první akce; *tail*(π) — zbytek; *sound*(π, I, B) — plán π je korektní vzhledem k záměrům *I* a přesvědčením *B*.

Plánovací funkce agenta: $plan : 2^{Bel} \times 2^{Int} \times 2^{Ac} \rightarrow Plan$.

Knihovna plánů. Agent nemusí konstruovat plány online — to je drahé. Velmi často je *plan()* implementována jako *knihovna hotových plánů*: stačí ji jednou projít a zkontrolovat, zda předpoklady plánu odpovídají aktuálnímu přesvědčení a efekty plánu odpovídají cíli. Tento nápad je jádrem architektury PRS (sekce 2.6).

2.4 Implementace BDI agenta

Hlavní řídicí smyčka BDI agenta:

```

1:  $B \leftarrow B_0; \quad I \leftarrow I_0$ 
2: while true do
3:    $v \leftarrow see()$ 
4:    $B \leftarrow brf(B, v)$ 
5:    $D \leftarrow options(B, I)$ 
6:    $I \leftarrow filter(B, D, I)$ 
7:    $\pi \leftarrow plan(B, I, Ac)$ 
8:   while  $\neg(empty(\pi) \vee succeeded(I, B) \vee impossible(I, B))$  do
9:      $a \leftarrow hd(\pi); \quad execute(a); \quad \pi \leftarrow tail(\pi)$ 
10:     $v \leftarrow see(); \quad B \leftarrow brf(B, v)$ 
11:    if reconsider( $I, B$ ) then
12:       $D \leftarrow options(B, I); \quad I \leftarrow filter(B, D, I)$ 
13:    end if
14:    if  $\neg sound(\pi, I, B)$  then
15:       $\pi \leftarrow plan(B, I, Ac)$  ▷ přeplánování
16:    end if
17:  end while
18: end while

```

Význam predikátů: *succeeded*(I, B) — záměry *I* platí za předpokladu přesvědčení *B*; *impossible*(I, B) — záměry *I* nemohou platit za předpokladu *B*. Vnitřní cyklus tedy končí, když je plán vyčerpán, cíl splněn, nebo se stal nedosažitelným.

2.5 Odhodlání agenta (*commitment*)

Strategie odhodlání vůči záměrům

Kdy má agent opustit cíl?

- **Slepé odhodlání** (*blind commitment*, též *fanatical*) — agent udržuje záměr, dokud *nezačne věřit, že ho dosáhl*. (Nikdy se nevzdá.)
- **Jednosměrné odhodlání** (*single-minded commitment*) — udržuje záměr, dokud *nezačne věřit, že ho dosáhl, nebo že už není dosažitelný*.
- **Otevřené odhodlání** (*open-minded commitment*) — záměr přetrvává, dokud agent věří, že je stále dosažitelný (a stále chtěný) — tedy může být opuštěn i při změně motivace.

Odhodlání vůči plánu versus **odhodlání vůči cíli**. Agent v algoritmu výše je odhodlán jednomu plánu; opustí ho, když věří, že cíl je splněn, že je nedosažitelný, nebo když je plán prázdný. Zbývá ale těžší otázka: kdy má agent *přehodnotit sám záměr*?

Klasické dilema: deliberace **trvá čas** a prostředí se během ní mění.

- Agent, který záměry *nepřehodnocuje*, se může upnout k cílům, jichž už nelze dosáhnout.
- Agent, který přehodnocuje *příliš často*, je tak zaneprázdněn uvažováním, že nic nestihne.

Metařízení (Wooldridge, Parsons, 1999): funkce *reconsider()*, která je *výpočetně jednodušší* než samotné přehodnocení. Kritérium kvality: *reconsider()* funguje dobře, pokud pokaždé, když navrhne přehodnocení, agent po deliberaci *skutečně* záměr změní (a naopak).

Odvážný vs. opatrný agent

Odvážný agent (*bold agent*) — přehodnocuje záměry až po dokončení celého plánu (nikdy plán nepřerušuje kvůli deliberaci).

Opatrný agent (*cautious agent*) — přehodnocuje po každé akci plánu.

Míra odvahy (*level of boldness*) — kolik akcí agent provede mezi dvěma deliberacemi.

Dynamičnost prostředí (*rate of world change*) — kolikrát se prostředí změní během jednoho cyklu agenta.

Efektivita agenta = počet dosažených záměrů / počet všech záměrů.

Experimentální výsledek (Kinny, Georgeff, v Tileworldu):

- Mění-li se svět *pomalu*, jsou efektivnější **odvážní** agenti — deliberace by byla zbytečná režie.
- Mění-li se svět *rychle*, překonávají je **opatrní** agenti — plány rychle zastarávají.

Toto je konkrétní kvantitativní podoba obecného kompromisu reaktivita vs. proaktivita.

2.6 Procedurální odvozovací systém (PRS)

PRS — *Procedural Reasoning System*

Georgeff a kol., 80. léta, Stanford. **První implementace architektury BDI** a jedna z prakticky nejúspěšnějších agentních architektur — byla mnohokrát reimplementována: **AgentSpeak/Jason, JAM, JACK, Jadex**.

Praktická nasazení: OASIS (systém řízení letového provozu v Sydney), SPOC (organizace podnikových procesů), SWARM (letecký simulátor australského letectva).

Komponenty PRS agenta: databáze *beliefs* (atomy FOL, prologovského typu), *desires* (cíle), *knihovna plánů* (*plan library*), zásobník *intentions* a *interpret*, který vše propojuje. Data ze senzorů vstupují do *beliefs*, *interpret* vydává akce.

Plán v PRS

Agent má knihovnu hotových plánů reprezentujících jeho *procedurální znalosti*. **Neplánuje se od nuly**, pouze se z knihovny *vybírá*. Plán má tři části:

- **Goal** (též *invocation condition* / *cue*) — podmínka, která má platit po vykonání plánu; určuje, na jaký cíl se plán hodí.
- **Context** — podmínka nutná pro spuštění plánu (musí být konzistentní s přesvědčeními).
- **Body** — akce, které se mají vykonat.

Tělo plánu *není* jen lineární posloupnost akcí — může obsahovat další *cíle*: „dosáhni *f*“, „dosáhni *f* nebo *g*“, „dosahuj *f*, dokud nenastane *g*“. Tím vzniká hierarchické rozkládání cílů na podcíle.

Běh interpretu:

1. Inicializace: počáteční přesvědčení B_0 a cíl nejvyšší úrovně (*top-level goal*).
2. Zásobník záměrů obsahuje aktuální cíle v různých stavech rozpracovanosti.
3. Interpret vezme záměr z vrcholu zásobníku a v knihovně plánů hledá plány s odpovídajícím cílem.
4. Z nalezených plánů vybere ty, jejichž *kontext* je konzistentní s aktuálními přesvědčeními — ty tvoří aktuální *options* / *desires*.
5. **Deliberace** = výběr jednoho záměru z možností:
 - Původní PRS používal *metaplány* — plány o plánech, které modifikovaly záměry agenta. Ukázalo se to příliš složité.
 - Praktičtější varianta: každý plán má číselné *ohodnocení* (očekávanou utilitu), vybere se plán s nejvyšší hodnotou.
6. Vybraný plán se vykoná, což může vést k přidání dalších záměrů na zásobník.
7. Selže-li plán, agent zvolí jiný záměr z možností a pokračuje.

Ukázka — Jason/AgentSpeak (agent nosí majiteli pivo z ledničky):

```
available(beer,fridge). limit(beer,10). /* počáteční beliefs */
too_much(B) :- .count(consumed(_,_ ,B),Q) & limit(B,L) & Q > L. /* pravidlo */
+!has(owner,beer) /* spouštěcí událost */
: available(beer,fridge) & not too_much(beer) /* kontext */
<- !at(robot,fridge); open(fridge); get(beer); close(fridge); /* tělo */
!at(robot,owner); hand_in(beer).
+!at(robot,P) : not at(robot,P) <- move_towards(P); !at(robot,P).
```

Zápis `!g : c <- b` čte se: „nastane-li událost přidání cíle *g* a platí-li kontext *c*, vykonej tělo *b*“. Symbol `!` označuje *achievement goal* (dosáhni), `+/-` přidání/odebrání přesvědčení nebo cíle, `.send` a `.count` jsou vestavěné akce.

2.7 Reaktivní přístup a jeho východiska

V 80. a 90. letech se ukázalo, že drobné úpravy symbolického přístupu nestačí, a vznikly alternativní paradigmaty UI. Jejich společným jmenovatelem je **odmítnutí symbolické reprezentace** a uvažování založeného na syntaktické manipulaci. Tři klíčové teze:

Tři principy reaktivního přístupu

Interakce (*situatedness*) — inteligentní chování závisí na prostředí, ve kterém je agent *zasazen*; nelze ho studovat izolovaně.

Ztělesnění (*embodiment*) — inteligentní chování není jen logika, je produktem agenta, který *má tělo* a fyzicky interaguje se světem.

Emergence — inteligentní chování *vzniká* (emerguje) interakcí mnoha jednoduchých chování; není nikde explicitně naprogramováno.

Charakteristiky reaktivních agentů: jsou *behaviorální* (důraz na vývoj a kombinaci jednotlivých chování), *situovaní* a *reaktivní* (agent v podstatě jen reaguje na prostředí, nedeliberuje). Reprezentace

je **subsymbolická**: konekcionistické sítě, konečné automaty, jednoduchá pravidla typu IF–THEN.

2.8 Symbolické reaktivní plánování: subsumpční architektura

Subsumpční architektura (*subsumption architecture*)

Rodney Brooks, 1986–1991; nejúspěšnější reaktivní přístup. Tři Brooksovy teze:

1. Inteligentní chování lze generovat *bez symbolické reprezentace*.
2. Inteligentní chování lze generovat *bez explicitního abstraktního uvažování*.
3. Intelligence je *emergentní vlastnost* určitých složitých systémů.

Intelligence agenta je realizována množinou jednoduchých cílově orientovaných **chování** (*behaviours*), pro něž platí:

- Každé chování je samo o sobě mechanismus výběru akce (dvojice *situace* $\rightarrow$ *akce*).
- Každé chování přijímá vjemy a transformuje je na akce, je odpovědné za jeden dílčí cíl a má jednoduchou strukturu pravidla.
- Chování běží *paralelně* a *soupeří* spolu o kontrolu nad agentem.
- Chování jsou uspořádána do **subsumpční hierarchie**, která definuje jejich priority (vyšší vrstva „pohlcuje“ = potlačuje nižší).

Mechanismus výběru akce v subsumpční architektuře:

```
1: function ACTION(vjem  $p$ )  
2:    $fired \leftarrow \{ b \in Behaviours \mid p \text{ splňuje podmínku } b \}$  ▷ chování, která „vystřelí“  
3:   for all  $b \in fired$  do  
4:     if neexistuje  $b' \in fired$  takové, že  $b' \prec b$  then ▷  $b$  není nikým inhibováno  
5:       return akce příslušná chování  $b$   
6:     end if  
7:   end for  
8:   return null ▷ nevystřelilo nic, agent nedělá nic  
9: end function
```

Formálně: chování je dvojice $\langle c, a \rangle$, kde $c \subseteq Per$ je podmínka a $a \in A$ akce; na množině chování R je definována relace inhibice $\prec \subseteq R \times R$ (striktní totální uspořádání). Pozor na směr — u Wooldridge i na přednášce znamená $b_1 \prec b_2$, že b_1 **inhibuje** b_2 , tedy b_1 je *níže* v subsumpční hierarchii a má *přednost*. Zápis $b_1 \prec b_2 \prec \dots \prec b_n$ tak vyjmenovává chování od nejvyšší priority k nejnižší.

Vlastnosti: mechanismus je *jednoduchý* (ale při desítkách chování bývají priority záludné) a *velmi rychlý* — lze ho implementovat hardwarově a má konstantní složitost.

Příklad: Steelsův průzkumník Marsu. Cílem je sbírat vzácné vzorky hornin na vzdálené planetě rojem robotických průzkumníků. Prostředky: základna vysílá navigační signál (roboti nemusí umět nic víc než detekovat *gradient* signálu); každý robot má radioaktivní drošky pro *nepřímou komunikaci* (stigmergii) s ostatními. Přímá komunikace není potřeba.

První iterace — náhodná procházka a návrat jednoho robota (priorita klesá shora dolů):

```
R1 if vidím překážku then změň směr  
R2 if nesu vzorek and jsem na základně then polož vzorek  
R3 if nesu vzorek and nejsem na základně then jdi po gradientu nahoru  
R4 if vidím vzorek then seber ho  
R5 if true then pohybuj se náhodně
```

Druhá iterace — lepší průzkum pomocí stigmergie. Vzorky se často vyskytují ve shlucích; robot, který jeden našel, o tom „řekne“ ostatním tím, že cestou domů rozhazuje drošky:

```
R6 if nesu vzorek and jsem na základně then polož vzorek  
R7 if nesu vzorek and nejsem na základně then polož 2 drošky a jdi po gradientu nahoru  
R8 if cítím drobek then seber 1 drobek a jdi po gradientu dolů
```

Priority (v notaci „levé inhibuje pravé“, tedy od nejvyšší priority): $R1 \prec R6 \prec R7 \prec R4 \prec R8 \prec R5$. Robot sledující stopu bere po jednom drobkem, takže se stopa postupně *vypaří*, jakmile je shluk vytěžen. Roj vykazuje kooperativní chování, přestože žádný robot nemá model světa ani neposílá zprávy.

2.9 Konekcionalistické reaktivní plánování: agent network architecture

Subsumpční architektura řeší konflikty *pevnou* prioritou, což je rigidní. Konekcionalistický přístup nahradí pevné priority **spojitou aktivací šířenou sítí** — výběr akce je výsledkem dynamiky sítě, ne pevného pořadí. Jde o „reaktivní plánování“: agent se chová, jako by plánoval, ale žádný explicitní plán nikdy nevytvoří.

Agent network architecture / spreading activation networks

Pattie Maes, 1989–1991. Agent je množina **kompetenčních modulů** (*competence modules*), podobných Brooksovým chováním. Každý modul i má:

- **předpoklady** c_i (*preconditions*) — co musí platit, aby byl modul *proveditelný* (*executable*);
- **efekty**: *add list* a_i a *delete list* d_i (jako u STRIPS);
- **úroveň aktivity** α_i a globální **práh aktivity** θ .

Moduly jsou propojeny v orientovaném grafu na základě svých podmínek — shoda pre- a postpodmínek tvoří hrany, k nimž přibývají hrany reprezentující časovou následnost a konflikty. V každém kroku se šíří aktivace; **vybrán je proveditelný modul s nejvyšší aktivací**, pokud překročí práh θ .

Tři zdroje a tři cesty šíření aktivity:

- **Ze stavu světa** (*state*): moduly, jejichž předpoklady jsou momentálně splněny, dostávají aktivaci — „teď se dá udělat tohle“.
- **Z cílů** (*goals*): moduly, jejichž *add list* obsahuje cíl, dostávají aktivaci — „tohle vede k cíli“.
- **Z ochranných cílů** (*protected goals*): moduly, jejichž *delete list* obsahuje již splněný cíl, jsou *inhibovány* — „tohle by zbořilo, co už mám“.

Aktivace se dále šíří mezi moduly třemi typy spojů:

- **Následnické spoje** (*successor links*): modul x posílá aktivaci modulu y , pokud y přidává něco, co x potřebuje jako předpoklad — *dopředné* řetězení („až budu hotov, půjde to dál“).
- **Předchůdcovské spoje** (*predecessor links*): modul x , který není proveditelný, posílá aktivaci modulům, které by mu splnily chybějící předpoklad — *zpětné* řetězení („pomoz mi, ať můžu běžet“).
- **Konfliktní spoje** (*conflict links*): modul x *inhibuje* modul y , který ruší jeho předpoklady.

Chování sítě řídí pět globálních parametrů: θ (práh aktivity, jediný práh v systému), ϕ (aktivace vpravená do sítě stavem světa), γ (aktivace vpravená cíli), δ (aktivace odebraná chráněnými cíli) a π (průměrná úroveň aktivity, na kterou se síť po každém kroku normalizuje, aby se celková energie neměnila). Poměr γ/ϕ přesně vyjadřuje kompromis **proaktivita vs. reaktivita**; θ určuje **uvážlivost vs. rychlost** (vysoký práh $\rightarrow$ agent déle váhá, ale rozhoduje se lépe). Pokud v daném kroku práh nepřekročí žádný proveditelný modul, θ se sníží o 10 % a zkouší se znovu — síť tak nemůže „zamrznout“.

Jak se aktivace v jednom kroku počítá. Aktualizace probíhá ve čtyřech fázích: (1) *vstup z prostředí* — každý pravdivý fakt rozdělí kvantum ϕ rovným dílem mezi moduly, které jej mají v předpokladech; (2) *vstup od cílů* — každý cíl rozdělí γ mezi moduly, které ho přidávají, a každý chráněný cíl *odebere* δ modulům, které by ho rušily; (3) *šíření mezi moduly* — proveditelný modul posílá část své aktivity dopředu následníkům (opět rozdělenou rovným dílem mezi příjemce), neproveditelný modul posílá aktivaci zpět modulům, které mu mohou splnit chybějící předpoklad, a konfliktní spoje odpovídající díl aktivity odečítají; (4) *normalizace* — aktivace všech modulů se přeškálují tak, aby

jejich průměr byl π (celková energie sítě zůstává konstantní). Poté se vybere proveditelný modul s aktivací nad prahem θ ; provedený modul svou aktivací vynuluje.

Poznámka k přednášce: slidy uvádějí práh aktivace jako vlastnost jednotlivého modulu (a mluví o něm jako o prioritě); v Maesově původním modelu je práh θ *globální* a roli „priority“ hraje aktuální úroveň aktivace α_i , která se dynamicky mění.

Srovnání se subsumpcí: pořadí chování není pevné, ale vzniká *dynamicky* z aktuálního stavu světa a cílů. Síť tak dokáže vygenerovat i víceukrokové „plánovité“ sekvence (aktivace se řetězí zpět od cíle přes předchůdcovské spoje), aniž by se kdy vytvořila explicitní datová struktura plánu — odtud název *reaktivní plánování*. Cenou je citlivost na nastavení parametrů a nemožnost zaručit optimalitu či úplnost.

2.10 Omezení reaktivních architektur

- Reaktivní agenti **nemají model světa**; vše musí odvodit z aktuálních vjemů. Někdy proto musí *měnit prostředí*, aby si „zapamatovali“ informaci (radioaktivní droby) — externalizovaná paměť, stigmergie.
- Mají jen **krátkodobý pohled**: jednají podle aktuálního stavu a lokální informace, je obtížné zohlednit globální podmínky a dlouhodobé cíle.
- Emergence **není dobrý inženýrský postup**: chování je třeba vyladit experimentálně, chybí metodologie i možnost verifikace.
- Návrh je zvládnutelný pro několik vrstev; s **desítkami chování se stává neřešitelným** (počet interakcí roste kvadraticky).

2.11 Hybridní architektury

Ani čistě reaktivní, ani čistě deliberativní architektura není ideální. Hybridní architektury kombinují obojí:

- **Deliberativní / plánovací komponenta** pracuje na symbolické úrovni, vytváří reprezentace a plány.
- **Reaktivní komponenta** zajišťuje okamžité reakce bez složitých výpočtů.

Komponenty jsou obvykle uspořádány do **vrstev** (*layered architectures*), přičemž reaktivní vrstvy mají zpravidla *přednost* před deliberativními (bezpečnost před optimalitou).

Horizontální vs. vertikální vrstvení

Horizontální vrstvení: všechny vrstvy jsou *paralelně* připojeny k senzorům i k efektorům. Každá vrstva je jako samostatný agent a navrhuje akci.

- + Koncepčně jednoduché: pro n druhů chování stačí n vrstev.
- Vrstvy se navzájem ovlivňují a jejich návrhy si odporují; je nutná **mediační funkce** (*mediator function*), která konflikt rozhodne. Ta je úzkým hrdlem: musí uvažovat m^n kombinací (n vrstev, m možných návrhů) a je kritickým bodem selhání.

Vertikální vrstvení: vrstvy jsou zapojeny *sériově*.

- *One-pass*: vjem vstoupí do nejnížší vrstvy, prochází nahoru, akce vystupuje z nejvyšší. Přirozené uspořádání a hierarchie chování.
- *Two-pass*: vjemy putují *zdola nahoru*, řídicí povely (akce) *shora dolů*. Tok připomíná řízení ve skutečné firmě.
- + Odpadá potřeba mediace, složitost interakcí je lineární ($n - 1$ rozhraní).
- Není robustní: selhání kterékoli vrstvy shodí celého agenta.

TouringMachines (Ferguson, 1992)

Horizontální třívrstvá architektura pro simulovaného agenta v dopravním prostředí.

- **Modelovací vrstva** (*modelling layer*) — symbolické modely agenta, prostředí a ostatních agentů; předvídá a řeší konflikty, stanovuje cíle a posílá je plánovací vrstvě.
- **Plánovací vrstva** (*planning layer*) — proaktivní chování; vybírá z předpřipravených plánů (podobně jako PRS) a sestavuje z nich plán cesty.
- **Reaktivní vrstva** (*reactive layer*) — klasická reaktivní pravidla *situace* → *akce*, rychlá okamžitá reakce (vyhýbání překážkám).

Vrstvy pracují paralelně a jejich výstupy sjednocuje sada **řídících pravidel** (*control rules*) — konkrétní podoba mediační funkce; ta mohou vstupy a výstupy vrstev potlačovat (*censor* a *suppressor* pravidla).

InteRRaP (Müller, 1994)

Vertikální dvouprůchodová (*two-pass*) třívrstvá architektura.

- **Vrstva kooperativního plánování** (*cooperative planning layer*) — sociální interakce, společné plány s ostatními agenty.
- **Vrstva lokálního plánování** (*local planning layer*) — běžné individuální plánování.
- **Behaviorální vrstva** (*behaviour-based layer*) — reaktivní chování.

Každá vrstva má *vlastní znalostní bázi* na příslušné úrovni abstrakce (znalosti o světě / plány / společné plány a modely ostatních agentů). Vrstvy komunikují dvěma operacemi: *aktivace zdola nahoru* (nižší vrstva neumí situaci zvládnout a předá ji výš) a *zásah shora dolů* (vyšší vrstva používá nižší k vykonání svých rozhodnutí).

Reálný příklad: Stanley (Volkswagen Touareg R5, Stanford). Autonomní vůz, který vyhrál DARPA Grand Challenge 2005 (212 km Mohavskou pouští); přímý předchůdce dnešních autonomních vozidel. Vrstvená architektura:

- vrstva senzorů → vrstva abstraktní percepce → vrstva plánování a řízení (plán trasy a řízení vozu) → vrstva rozhraní vozidla;
- plus vrstva uživatelského rozhraní (panel, start) a vrstva globálních služeb (souborový systém, komunikace, hodiny).

2.12 Komplexní architektury: IDA a globální pracovní prostor

Poslední rodina architektur jde opačným směrem než reaktivní minimalismus: snaží se do jednoho agenta integrovat *všechno* — výběr akce, paměť, deliberaci, emoce.

IDA — *Intelligent Distribution Agent* (S. Franklin)

Jedna z nejsložitějších agentních architektur, jaké byly postaveny. Vznikla pro reálnou úlohu amerického námořnictva — přidělování personálu na nová služební místa (odtud „distribution“); distribuovaná je i sama architektura.

Východisko: teorie globálního pracovního prostoru (*global workspace theory*, Baars, 1988–97) — jedna z teorií vědomí:

- Mysl je multiagentní systém: skládá se z mnoha jednoduchých, převážně nevědomých procesů provádějících specializované operace.
- Procesy spolu komunikují *zřídka* a pouze přes sdílenou paměť typu *blackboard*, organizovanou jako asociativní pole (srov. kap. 5).
- Procesy dynamicky vytvářejí **koalice** vyšších řádů.
- **Mechanismus výběru akce:** koalice, která nejlépe odpovídá aktuálním vstupům, se dostane do „vědomí“ (globálního pracovního prostoru) a je vykonána.
- Existuje *hierarchie kontextů* reprezentujících svět na různých úrovních — kontext cílů, vjemů, smyslů, konceptů, kulturní kontext.

Hodnocení: kombinuje mnoho přístupů z MAS i zbytku UI (výběr akce, paměť, deliberace, emoce), a právě to je jeho slabina — řada rozhodnutí je *ad hoc* a sladit tak heterogenní kolekci modelů do optimálně spolupracujícího celku je velmi obtížné.

Pro srovnání mechanismů výběru akce je IDA užitečný jako *čtvrtý* typ arbitráže: subsumpce rozhoduje **pevnou prioritou**, síť Maes **úrovni aktivace**, vrstvené architektury **mediační funkcí**, IDA **soutěží dynamicky vznikajících koalic** o přístup do globálního pracovního prostoru.

2.13 Srovnání architektur

	Deduktivní	BDI / PRS	Reaktivní	Hybridní
Reprezentace	symbolická (FOL)	symbolická (beliefs, plány)	subsymbolická / pravidla	smíšená po vrstvách
Výběr akce	dokazování $Do(a)$	deliberace + plán z knihovny	priorita / aktivace	vrstvy + arbitráž
Model světa	úplný	částečný, revidovatelný	žádný	po vrstvách
Rychlost	velmi nízká	střední	vysoká (konst.)	vysoká v reflexu
Reaktivita	špatná	střední (laditelná)	výborná	dobrá
Proaktivita	dobrá	výborná	žádná	dobrá
Návrh	obtížný, ale formální	systematický	experimentální	složitý

2.14 Shrnutí ke zkoušce

- **Deduktivní agent:** $DB \in 2^L$, $DB \vdash_{\rho} Do(a)$; dvoufázový výběr akce (dokazatelná akce $\rightarrow$ konzistentní akce $\rightarrow$ nic). Elegantní, ale nepraktický; problém transdukce a vypočitatelné racionality.
- **BDI:** dvě fáze — deliberace (*brf*, *options*, *filter*) a means-ends reasoning (plánování). Beliefs jsou subjektivní a omylné, desires mohou být nekonzistentní, intentions jsou závazky, které přetrvávají a omezují další deliberaci.
- **STRIPS:** deskriptor akce $\langle P_a, D_a, A_a \rangle$, plánovací problém $\langle B_0, O, G \rangle$, $B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}$; plán *přípustný* ($B_{i-1} \models P_{a_i}$) vs. *korektní* (navíc $B_n \models G$). Umět projít svět kostek.
- **BDI smyčka** a role *reconsider* a *sound*.
- **Commitment:** blind / single-minded / open-minded; bold vs. cautious agent; efektivita = dosažené/všechny záměry; pomalé prostředí $\rightarrow$ bold, rychlé $\rightarrow$ cautious.
- **PRS:** první implementace BDI; plán = Goal + Context + Body; knihovna plánů místo plánování; zásobník záměrů; deliberace metaplány nebo utilitou; potomci AgentSpeak/Jason, JAM, JACK, Jadex.
- Reaktivní přístup stojí na trojici *situatedness* — *embodiment* — *emergence*; odmítá symbolickou reprezentaci.
- **Subsumpce** (Brooks) = *symbolické* reaktivní plánování: množina jednoduchých chování *situace* $\rightarrow$ *akce*, běžících paralelně, uspořádaných do pevné priority. Umět Steelsova průzkumníka Marsu a roli stigmergie (drobky).
- **Agent network architecture** (Maes) = *konekcionistické* reaktivní plánování: moduly s předpoklady a efekty (add, delete), aktivace se šíří následnickými, předchůdcovskými a konfliktními spoji, zdroje aktivace jsou stav světa, cíle a inhibice od chráněných cílů; vybere se proveditelný modul s nejvyšší aktivací nad prahem. Parametry γ/ϕ ladí proaktivitu vs. reaktivitu.
- Omezení reaktivních agentů: bez modelu světa, krátkodobý pohled, neinženýrský návrh, neškálovatelné množství vrstev.
- **Hybridní:** horizontální (paralelní vrstvy + mediátor, m^n kombinací, úzké hrdlo) vs. vertikální (sériové, one-pass/two-pass, $n - 1$ rozhraní, křehké).
- **TouringMachines** = horizontální 3 vrstvy (modelling, planning, reactive) + řídicí pravidla. **InteRRaP** = vertikální two-pass 3 vrstvy (kooperativní plánování, lokální plánování, behaviorální) + vlastní KB na vrstvu. **Stanley** jako reálné nasazení.
- **IDA** (Franklin) + *global workspace theory* (Baars): mysl jako MAS jednoduchých nevědomých

procesů komunikujících přes blackboard; koalice procesů soutěží o vstup do „vědomí“ a vítězná se vykoná; hierarchie kontextů. Nejsložitější architektura, mnoho ad hoc rozhodnutí.

- Čtyři typy arbitráže mezi chováními: **pevná priorita** (subsumpce), **úroveň aktivace** (Maes), **mediační funkce** (vrstvené), **soutěž koalic** (IDA).

3 Problém hledání cesty

[♦ nad rámec slidů: celá kapitola] Přednáška NAIL106 uvádí plánování i robotiku mezi tématy, která *nepokrývá*, a slidy k hledání cesty nic nemají. Okruh ho ale výslovně jmenuje, takže je nutné ho umět. Kapitola je proto držena na minimu, které na zkoušce obstojí: A* s vlastnostmi heuristiky, a pak už jen to, co je na hledání cesty *multiagentního* — dynamické prostředí a koordinace více agentů.

3.1 Formulace a zařazení

Hledání cesty (*pathfinding*) je nejčastější konkrétní podúloha, kterou musí situovaný agent řešit — jako součást plánovací vrstvy hybridní architektury, jako akce `move_towards(P)` v BDI plánu, nebo jako služba poskytovaná jiným agentům. V multiagentním kontextu přibývají dvě specifiky: prostředí je **dynamické** (mapa se mění) a ostatní agenti jsou **pohyblivé překážky**, se kterými je nutné se koordinovat.

Problém hledání cesty

Je dán ohodnocený graf $G = (V, E)$ s nezáporným ohodnocením hran $w : E \rightarrow \mathbb{R}_0^+$, počáteční vrchol $s \in V$ a cílový vrchol $g \in V$. Hledáme cestu $s = v_0, v_1, \dots, v_k = g$ minimalizující $\sum_i w(v_{i-1}, v_i)$.

Graf bývá zadán *implicitně* nástupnickou funkcí (mřížka 4-/8-souvislá, navigační síť, graf viditelnosti).

Odkud se vezmou vrcholy. Prostředí je spojitě, prohledávání potřebuje graf — volba diskretizace rozhoduje o kvalitě cesty víc než volba algoritmu:

- **Mřížka** (4- nebo 8-souvislá) — triviální na implementaci, ale cesta je „schodovitá“ a vrcholů je mnoho.
- **Graf viditelnosti** — vrcholy jsou rohy mnohoúhelníkových překážek, hrany spojují vzájemně viditelné rohy. V rovině s mnohoúhelníkovými překážkami dá *skutečně optimální* cestu, ale roste kvadraticky s počtem rohů.
- **Navigační síť** (*navmesh*) — prostor se pokryje konvexními plochami, uvnitř nichž je pohyb volný; standard v herních enginech a robotice.
- **Stavová mřížka** — vrchol není jen poloha, ale i orientace (a případně rychlost); nutné pro neholonomní vozidla, která se nemohou otáčet na místě.
- **Vzorkovací metody** — *PRM* (*probabilistic roadmap*, předpočítaný graf náhodných konfigurací) a *RRT/RRT** (inkrementálně rostoucí strom). Jediná praktická volba pro vysokodimenzionální konfigurační prostory (rameno robota).
- **Potenciálová pole** — cíl přitahuje, překážky odpuzují, agent jde po gradientu. Velmi levné a reaktivní (patří spíš do reaktivní vrstvy, §2.8), ale *uvíznou v lokálním minimu* a nedávají žádnou záruku úplnosti.

3.2 A*

Prohledávání typu *best-first* udržuje pro každý vrchol n hodnoty

$g(n)$ = cena nejlepší dosud nalezené cesty $s \rightarrow n$,

$h(n)$ = heuristický odhad ceny $n \rightarrow g_{\text{cil}}$,

$f(n) = g(n) + h(n)$ — odhad ceny nejlepší cesty $s \rightarrow g_{\text{cil}}$ vedoucí přes n .

Algoritmus A* (Hart, Nilsson, Raphael, 1968):

```
1:  $OPEN \leftarrow \{s\}$  (prioritní fronta podle  $f$ ),  $CLOSED \leftarrow \emptyset$ 
2:  $g(s) \leftarrow 0$ ,  $f(s) \leftarrow h(s)$ , ostatní  $g \leftarrow \infty$ 
3: while  $OPEN \neq \emptyset$  do
4:    $n \leftarrow \arg \min_{x \in OPEN} f(x)$ ; odeber  $n$  z  $OPEN$ 
5:   if  $n$  je cíl then return rekonstruovaná cesta
6:   end if
7:   přidej  $n$  do  $CLOSED$ 
8:   for all  $m \in succ(n)$  do
9:      $g_{\text{new}} \leftarrow g(n) + w(n, m)$ 
10:    if  $g_{\text{new}} < g(m)$  then
11:       $g(m) \leftarrow g_{\text{new}}$ ;  $parent(m) \leftarrow n$ ;  $f(m) \leftarrow g(m) + h(m)$ 
12:      vlož/aktualizuj  $m$  v  $OPEN$  (a odeber z  $CLOSED$ , byl-li tam)
13:    end if
14:  end for
15: end while
16: return „cesta neexistuje“
```

Přípustnost a konzistence heuristiky

Heuristika h je **přípustná** (*admissible*), pokud nikdy nenadhadnocuje: $h(n) \leq h^*(n)$ pro všechna n , kde $h^*(n)$ je skutečná cena nejlevnější cesty z n do cíle.

Heuristika je **konzistentní** (*consistent*, monotónní), pokud pro každou hranu (n, m) platí trojúhelníková nerovnost

$$h(n) \leq w(n, m) + h(m), \quad h(g_{\text{cil}}) = 0.$$

Vlastnosti: konzistence $\Rightarrow$ přípustnost. Při přípustné heuristice je A* *optimální* (najde nejlevnější cestu). Při konzistentní heuristice je navíc f podél každé cesty neklesající a každý vrchol je expandován *nejvýše jednou*.

Speciální případy a varianty: pro $h \equiv 0$ degeneruje A* na **Dijkstrův algoritmus**. Typické heuristiky na mřížce jsou manhattanská $|\Delta x| + |\Delta y|$ (4 směry), oktílová (8 směrů) a eukleidovská.

Weighted A* používá $f = g + \varepsilon h$, $\varepsilon > 1$ — je rychlejší, ale nalezená cesta je nejvýše $\varepsilon \times$ dražší; **IDA*** šetří paměť ($O(d)$ místo exponenciální).

Zařazení mezi prohledávací algoritmy. A* je jeden bod na škále, na jejíchž koncích stojí dvě krajnosti: *BFS* a *Dijkstra/uniform-cost* berou v úvahu jen dosavadní cenu g (úplně a optimální, ale prohledávají všemi směry stejně), zatímco *greedy best-first* bere jen odhad h (rychlé, ale *neoptimální* a snadno se dá svést). A* obojí kombinuje a je optimální při přípustné h . Obousměrné prohledávání (současně od startu a od cíle) řádově zmenšuje prohledaný objem.

Zrychlení bez ztráty optimality. Na mřížkách se používá **JPS** (*jump point search*): mřížka má obrovské množství stejně dlouhých cest lišících se jen pořadím kroků, JPS tuto symetrii odstraní tím, že přeskakuje „nezajímavé“ vrcholy a expanduje jen *skokové body*. **Theta*** a další *any-angle* algoritmy uvolňují požadavek, aby cesta vedla po hranách mřížky — při expanzi zkusí napojit vrchol přímo na *dědečka*, je-li mezi nimi přímá viditelnost, a dají tak podstatně kratší a přirozenější cesty.

HPA* (*hierarchical*) rozdělí mapu na clustery, předpočítá cesty mezi jejich vstupy a hledá nejdřív hrubě mezi clustery a teprve pak uvnitř nich.

Malý příklad. Mřížka 3×3 , pohyb do 4 směrů, cena hrany 1, start $(0, 0)$, cíl $(2, 2)$, překážka na $(1, 1)$; manhattanská heuristika $h(x, y) = |2 - x| + |2 - y|$ je přípustná i konzistentní. Start má $f = 0 + 4 = 4$; nástupci $(1, 0)$ a $(0, 1)$ mají $g = 1, h = 3, f = 4$; dále $(2, 0)$ a $(0, 2)$ s $g = 2, h = 2, f = 4$, pak $(2, 1)$ a $(1, 2)$ s $g = 3, h = 1, f = 4$ a konečně cíl $(2, 2)$ s $g = 4, h = 0, f = 4$. Všechny vrcholy na optimální cestě mají $f = 4$ — typický projev konzistentní heuristiky, která zbytek cesty odhaduje přesně.

3.3 Dynamické prostředí: D* Lite

Agent se pohybuje a průběžně zjišťuje, že mapa je jinde jiná (nová překážka, změněná cena). Přepřátovat A^* od nuly v každém kroku je drahé; řešením jsou **inkrementální** algoritmy, které při lokální změně přepočítají jen zasaženou část.

LPA* a D* Lite

LPA* (*Lifelong Planning A**, Koenig, Likhachev, 2001) — inkrementální A^* pro pevný start a cíl. Ke každému vrcholu udržuje kromě $g(n)$ i *lookahead* hodnotu

$$rhs(n) = \begin{cases} 0 & n = s, \\ \min_{p \in pred(n)} (g(p) + w(p, n)) & \text{jinak.} \end{cases}$$

Vrchol je **lokálně konzistentní**, pokud $g(n) = rhs(n)$. Změní-li se cena hrany, stanou se některé vrcholy nekonzistentními, zařadí se do prioritní fronty a změna se propaguje jen tam, kde je to potřeba.

D* Lite (Koenig, Likhachev, 2002) — LPA* prohledávaný *od cíle ke startu* (aby hodnoty zůstaly použitelné, když se robot posune), s korekcí prioritní fronty při pohybu. Funkčně ekvivalentní staršímu **D*** (Stentz, 1994), ale výrazně jednodušší; *de facto* standard v mobilní robotice.

V neznámém prostředí se přitom používá *optimistický předpoklad* (*freespace assumption*): neznámé buňky se považují za průchodné, agent jde po naplánované cestě, a jakmile narazí, přeplánuje. To je přesně scénář, pro nějž byl D^* navržen.

3.4 Hledání cest pro více agentů (MAPF)

♦ nad rámec slidů: není ve slidech, ale je to jediná vysloveně multiagentní část okruhu

Multi-Agent Path Finding, MAPF

Vstup: graf $G = (V, E)$ a k agentů, agent i má start s_i a cíl g_i . Čas je diskrétní; v každém kroku každý agent buď přejde po hraně, nebo počká na místě.

Konflikty — řešení musí být *bezkonfliktní*: *vrcholový konflikt* (dva agenti jsou ve stejném čase ve stejném vrcholu) a *hranový* / „*swap*“ *konflikt* (dva agenti si v jednom kroku vymění pozice).

Účelová funkce: *sum-of-costs* $\sum_i cost_i$, nebo *makespan* $\max_i cost_i$. Nalezení *optimálního* řešení je NP-těžké.

Prioritizované plánování (rozvázaný přístup). Agenti se uspořádají podle priority a pro $i = 1, \dots, k$ se cesta agenta i hledá algoritmem A^* v *prostoru-čase* (stav = dvojice ⟨vrchol, čas⟩) tak, aby se vyhnul již naplánovaným trajektoriím uloženým v **rezervační tabulce**. Varianta se nazývá *Cooperative A** (Silver, 2005). Je rychlá a škáluje, ale je **neúplná** — existují řešitelné instance, které neřeší žádné pořadí priorit (dva agenti v úzkém koridoru, kde jeden musí uhnout do výklenku).

Conflict-Based Search (CBS)

Sharon a kol., 2012–2015. Dvouúrovňový *optimální* a *úplný* algoritmus, který se vyhýbá exponenciálnímu sdruženému stavovému prostoru V^k .

Nízká úroveň: pro jednoho agenta se hledá optimální cesta v prostoru–čase pomocí A^* při respektování množiny *omezení* tvaru $\langle a_i, v, t \rangle$ („agent i nesmí být ve vrcholu v v čase t “).

Vysoká úroveň: prohledává *strom omezení* podle ceny. Kořen má prázdnou množinu omezení a individuálně optimální cesty. Vezme se uzel s nejnižší cenou; jsou-li cesty bezkonfliktní, je řešení optimální. Jinak se vybere první konflikt a uzel se *rozvětví* na dva potomky — do každého se přidá omezení pro jednoho z kolidujících agentů a přeplánuje se *jen* on.

Škálovatelné varianty. Optimální CBS zvládá desítky agentů; pro stovky až tisíce (sklady Amazon/Kiva) se optimalita obětuje kontrolovaným způsobem: *ECBS* (*bounded suboptimal* — na obou úrovních se používá *focal search* a výsledek je zaručeně nejvýše $w \times$ horší než optimum), *SIPP* (*safe interval path planning* — místo jednotlivých časových kroků se pracuje se *souvislými intervaly*, v nichž je vrchol volný, takže stavový prostor nezávisí na délce horizontu) a *MAPF-LNS* (*large neighborhood search* — opakovaně se zahodí a znovu naplánují cesty malé podmnožiny agentů).

3.5 Shrnutí ke zkoušce

- A^* : $f = g + h$; *přípustnost* $h \leq h^* \Rightarrow$ optimalita; *konzistence* $h(n) \leq w(n, m) + h(m) \Rightarrow$ každý vrchol se expanduje jednou. $h \equiv 0$ dá Dijkstru; weighted A^* je ε -suboptimální.
- Dynamické prostředí: LPA^* (hodnoty g a rhs , lokální konzistence $g = rhs$), D^* Lite (prohledávání od cíle, inkrementální oprava), freespace assumption.
- MAPF: vrcholové a hranové (swap) konflikty, sum-of-costs vs. makespan; optimální MAPF je NP-těžký.
- Prioritizované plánování / Cooperative A^* s rezervační tabulkou — rychlé, ale neúplné. **CBS** — nízká úroveň A^* s omezeními, vysoká úroveň větvení stromu omezení podle konfliktu; optimální a úplný. Škálovatelně: ECBS, SIPP, LNS.
- Diskretizace prostoru rozhoduje o kvalitě cesty víc než algoritmus: mřížka, graf viditelnosti (optimální v rovině s mnohoúhelníky), navmesh, stavová mřížka, PRM/RRT (vysoké dimenze), potenciálová pole (levná, uvíznou v lokálním minimu).
- Zařazení: BFS/Dijkstra berou jen g , greedy best-first jen h , A^* obojí. Zrychlení bez ztráty optimality: JPS (symetrie v mřížce), Theta* (any-angle), HPA* (hierarchie).

4 Komunikace a znalosti v MAS: ontologie, řečové akty, FIPA-ACL, protokoly

4.1 Od agenta k multiagentnímu systému

Umíme navrhnout jednoho agenta. Aby agenti tvořili systém, musí spolu komunikovat, a k tomu potřebují vyřešit:

- **reprezentaci znalostí** — v čem jsou znalosti vyjádřeny;
- **společný slovník** — aby stejné slovo znamenalo totéž (to je úloha *ontologií*);
- **standardní zprávy** — syntaxi a sémantiku komunikace (*ACL*);
- **předvídatelné chování při komunikaci** — *interakční protokoly*;
- **technické problémy** — distribuovanost, mobilita agentů, asynchronnost, nespolehlivé komunikační kanály.

4.2 Co je ontologie

Ontologie

Filozoficky (z řeckého *to on* = jsoucí, *logos* = slovo): nauka o bytí, podle Aristotela „první filozofie“, součást metafyziky.

V informatice: **explicitní a formalizovaný popis části reality**. Obvykle formální deklarativní popis obsahující *glosář* (definice konceptů) a *tezaurus* (definice vztahů mezi koncepty). Ontologie je jakýsi slovník pro ukládání a výměnu znalostí o doméně standardním způsobem.

Gruber (1995): „Ontologie je popis (podobný formální specifikaci programu) konceptů a vztahů, které mohou pro agenta nebo komunitu agentů formálně existovat.“ Často citovaná zkrácená verze: *ontologie je explicitní specifikace konceptualizace*.

Motivační příklad (dialog dvou lidí — nebo dvou agentů). „Slyšel jsi 7777? — Ne, co to je? — To je nové CD kapely SRPR, takový alt.rock s elektronikou a skvělými texty. — Aha.“ Aby si rozuměli, musí sdílet: 7777 je album; spadá do žánrů alt.rock a electronica; má skladby, interpreta (SRPR), autory hudby a textů; SRPR je kapela, má členy, ti hrají na nástroje. . .

Formalismus ontologie

- **Třídy** (*classes*, koncepty) — věci mající něco společného.
- **Instance** (*individuals*, objekty) — konkrétní jedinci ze tříd.
- **Vlastnosti** (*properties*) — atributy tříd a instancí.
- **Relace mezi třídami**, zejména **podtřída** (*is-a*) — *tranzitivní* relace.
- Další relace a vlastnosti (základní znalosti o doméně).

Strukturální část (třídy, relace, axiomy) se obvykle nazývá vlastní *ontologie*; spolu s **fakty o konkrétních jedincích** tvoří **znalostní bázi** (*knowledge base*).

Ontologie ontologií podle síly formalizace (od slabých k silným): slovník (řízený slovník vybraných termínů) → glosář (definice významů v přirozeném jazyce) → tezaurus (synonyma) → neformální hierarchie (Amazon, wiki) → formální *is-a* hierarchie (subsumpce tříd) → třídy s vlastnostmi → omezení hodnot („každý člověk má právě jednu matku“) → libovolná logická omezení (vede na složité odvozovací algoritmy).

Ontologie ontologií podle rozsahu:

- **Aplikační** — nejběžnější v praxi, těžko znovupoužitelné.
- **Doménové** — oblíbený předmět výzkumu sémantického webu.
- **Horní** (*upper ontologies*) — měly by popisovat „všechno“ (Thing, Living thing, . . .) a sloužit jako základ doménových ontologií: BFO, GFO, UFO; WordNet a IDEAS nejsou vhodné pro formální odvozování. Vůči snaze o univerzální ontologii existují metodologické námitky (Wittgenstein, *Tractatus logico-philosophicus*).

4.3 Ontologické jazyky

Ontologické jazyky jsou formální deklarativní jazyky pro reprezentaci znalostí; obsahují fakta a odvozovací pravidla a nejčastěji se opírají o logiku prvního řádu nebo o *deskripční logiku*.

Rámce (*frames*) — historický předchůdce (Minsky), oblíbený v klasické UI a expertních systémech. Odpovídají objektovému programování:

Terminologie rámců	Objektová terminologie
Frame (rámec)	třída objektů
Slot	vlastnost / atribut objektu
Trigger, accessor, mutator	metoda

XML nebyl pro ontologie vytvořen, ale používá se: hlavní výhodou je možnost definovat vlastní

značky, které přirozeně reprezentují slovník:

```
<product typ="CD">
  <title>7777</title>
  <artist>SRPR</artist>
</product>
```

Chybí mu ovšem definovaná sémantika — popisuje jen strukturu dokumentu, ne význam.

RDF — *Resource Description Framework*

Standardní nástroj pro reprezentaci znalostí (nejen) pro web. Je **jednoduchý**: málo expresivní, zato s rychlými odvozovacími algoritmy.

Vše je vyjádřeno **trojicemi** *subjekt – predikát – objekt*, tedy *Predikt(Subjekt, Objekt)*. Například *MarriedTo(Karel, Jja)*, *FatherOf(Karel, Pa)*. Množina trojic tvoří orientovaný ohodnocený graf (*RDF graph*); zdroje se identifikují pomocí IRI. Serializace: RDF/XML, Turtle, N-Triples, JSON-LD. Dotazovacím jazykem je SPARQL. Nadstavba **RDFS** přidává třídy, `subClassOf`, `domain`, `range`.

OWL — *Web Ontology Language*

Rovněž pochází ze (sémantického) webu, existuje ve verzích 1 a 2. Je to *kolekce několika formalismů* pro popis ontologií; syntaxe může být XML, RDF, funkcionální notace, Manchester syntax nebo Turtle. Snaha o formální, a přesto prakticky použitelný jazyk.

Základem je **deskripční logika** — *rozhodnutelný fragment* logiky prvního řádu.

Předpoklad otevřeného světa (*open world assumption*, OWA): co nevíme, je *nedefinované* — na rozdíl od předpokladu uzavřeného světa (databáze, Prolog), kde je vše, co nevíme, *nepravdivé*.

Deskripční logika

Tři druhy entit: **koncepty** (třídy), **role** (vlastnosti, predikáty) a **individua** (objekty). **Axiom** je logický výraz o rolích a konceptech — na rozdíl od rámců a objektového programování, které třídu popisují *úplně*, axiomy popisují třídu jen *částečně* (a mohou přicházet z více zdrojů).

Jde o celou rodinu logických systémů podle povolených konstruktorů. Základní **ALC**: negace, konjunkce a disjunkce konceptů, omezené existenční a univerzální kvantifikátory. Rozšíření: hierarchie rolí (*H*), tranzitivita (*S*), inverzní role (*I*), kardinalitní omezení (*N*, *Q*), nominály (*O*). Znalostní báze se dělí na **T-Box** (terminologie — axiomy o konceptech a rolích) a **A-Box** (asertivní část — fakta o individuích).

Profilý OWL. *OWL 1 Lite (SHIF)* — nejjednodušší, blízko RDF, mnoho omezení kvůli rychlému strojovému zpracování; *OWL 1 DL (SHOIN)* — odpovídá deskripční logice (umožňuje např. vyjádřit disjunktnost tříd), je úplný a rozhodnutelný; *OWL 1 Full* — největší vyjadřovací síla, ale mnoho problémů je nerozhodnutelných. *OWL 2 (SROIQ)* přidává profily: *EL* (odvozování v polynomiálním čase, velké lékařské ontologie typu SNOMED), *QL* (dotazy nad znalostní bází, napojení na relační databáze) a *RL* (axiomy ve tvaru pravidel).

KIF — *Knowledge Interchange Format*

Reprezentace znalostí v logice prvního řádu, LISPovská prefixová syntaxe. Byl navržen v rámci projektu DARPA *Knowledge Sharing Effort* jako **vnitřní jazyk** zpráv (obsah), zatímco KQML byl *vnějším* jazykem (obálka). Příklady:

```
(salary 015-46-3946 widgets 72000)
(> (* (width chip1) (length chip1)) (* (width chip2) (length chip2)))
(forall (?F ?T) (= > (and (instance ?F Farmer) (instance ?T Tractor)) (likes
?F ?T)))
(exists (?F ?T) (and (instance ?F Farmer) (instance ?T Tractor) (likes ?F
?T)))
```

DAML+OIL — DARPA Agent Markup Language + Ontology Interchange Language, přímý předchůdce OWL, opuštěn v roce 2006.

4.4 Uvažování o znalostech: epistemická logika

♦ nad rámec slidů: slidy modální logiky výslovně vynechávají; okruh ale mluví o „znalostech v MAS“, takže minimum se hodí umět]

Epistemická logika a společná znalost

Znalost agenta se modeluje modálním operátorem $K_i\varphi$ („agent i ví, že φ “) s **kripkovskou sémantikou možných světů**: $K_i\varphi$ platí ve světě w , právě když φ platí ve *všech* světech, které agent i neumí od w odlišit. **Znalost** se axiomatizuje systémem **S5**; klíčový je axiom **T**: $K_i\varphi \Rightarrow \varphi$ (co vím, to platí). **Přesvědčení** (operátor B_i , srov. BDI) se axiomatizuje systémem **KD45**, kde axiom **T** nahrazuje slabší **D**: $\neg B_i \perp$ — přesvědčení musí být konzistentní, ale *může být mylné*. Rozdíl znalost vs. přesvědčení je tedy formálně právě axiom **T**.

Pro skupinu G : $E_G\varphi$ — každý v G ví φ ; **společná znalost** $C_G\varphi \equiv E_G\varphi \wedge E_GE_G\varphi \wedge \dots$ (všichni vědí, že všichni vědí, ...).

Koordinovaný útok (problém dvou generálů): dva generálové se přes nespolehlivého posla domlouvají na současném útoku. Žádný konečný počet potvrzení nevytvoří společnou znalost — po k zprávách platí nejvýše $E_G^k\varphi$, nikdy $C_G\varphi$. Poučení pro MAS: *nespolehlivá komunikace nikdy nevytvoří společnou znalost*, a tedy ani dokonalou koordinaci.

4.5 Proč je komunikace agentů jiná

V objektovém programování znamená „komunikace“ volání metody objektu s parametry — volaný objekt *musí* vyhovět. Agenti nemohou přinutit jiného agenta, aby něco udělal, ani nemohou přímo měnit jeho vnitřní proměnné. Musí **provést komunikační akt** s cílem

- vyměnit si informace, nebo
- *ovlivnit* jiné agenty, aby něco udělali.

Ostatní agenti mají vlastní cíle a je jen na nich, jak s přijatou informací, žádostí či dotazem naloží. To je zásadní rozdíl: **komunikace v MAS je akce, ne volání**.

4.6 Teorie řečových aktů

Řečové akty (*speech acts*) — Austin, 1962

Některé části užívání jazyka mají charakter **akcí**, protože mění stav světa podobně jako fyzické akce — to jsou řečové akty:

- „Prohlašuji vás mužem a ženou.“
- „Vyhlašuji válku.“

Jde o *pragmatickou* teorii jazyka: jak se řeč používá k dosahování cílů. Typická performativní slovesa: *request, inform, promise*.

Tři vrstvy řečového aktu

- **Lokuční akt** (*locutionary act*) — *co bylo řečeno*; samotný výrok. „Udělej mi čaj.“
- **Ilokuční akt** (*illocutionary act*) — *co bylo míněno*; lokuce + performativní význam (dotaz, žádost, ...). „Požádal mě o čaj.“
- **Perlokuční akt** (*perlocutionary act*) — *co se skutečně stalo*; efekt řečového aktu. „Přiměla mě, abych jí uvařil čaj.“

Pro MAS je klíčová **ilokuční** část — ta je obsahem zprávy (performativa).

Searle rozpracoval podmínky, za nichž je řečový akt „zdařilý“. Pro akt *MLUVČÍ požádá POSLUCHAČE o akci*:

- **Standardní vstupně-výstupní podmínky**: posluchač slyší, nejde o scénu ve filmu. . .
- **Předpoklady** (co musí platit, aby mluvčí mohl tento akt zvolit): posluchač je schopen akci provést; mluvčí věří, že je schopen ji provést; není zřejmé, že by ji posluchač provedl i bez požádání.
- **Upřímnost** (*sincerity*): mluvčí opravdu chce, aby se akce provedla.

Searlovy kategorie řečových aktů

Starší dělení: *request, advice, statement, promise, . . .*

Novější (a standardní) dělení do pěti tříd:

- **Asertiva** (*assertives*, reprezentativa) — informují posluchače (*inform*).
- **Direktiva** (*directives*) — žádají posluchače o akci (*request*).
- **Komisiva** (*commissives*) — mluvčí se k něčemu zavazuje (*promise*).
- **Expresiva** (*expressives*) — mluvčí vyjadřuje mentální stav, emoci („děkuji“).
- **Deklarace** (*declarations*) — mění stav věcí (válka, sňatek).

Řečový akt má vždy dvě složky: **performativní sloveso** (*request, query, inform*) a **propoziční obsah** („okno je zavřené“).

Plánovací teorie řečových aktů (Cohen, Perrault, 1979)

„... modelování [řečových aktů] v plánovacím systému jako operátorů definovaných... pomocí přesvědčení a cílů mluvčích a posluchačů. S řečovými akty se tak zachází stejně jako s fyzickými akcemi.“

Sémantika řečových aktů je tedy dána v duchu **STRIPS** (předpoklady – delete – add), jen se používají modální operátory pro *beliefs, abilities a wants*.

Příklad: Request a Inform.

	Request(S,H,A)	Inform(S,H,φ)
Předpoklad	$(S \text{ believe } (H \text{ cando } A)) \wedge$	$(S \text{ believe } \varphi)$
<i>cando</i>	$(S \text{ believe } (H \text{ believe } (H \text{ cando } A)))$	
Předpoklad <i>want</i>	$(S \text{ believe } (S \text{ want requestInstance}))$	$(S \text{ believe } (S \text{ want informInstance}))$
Efekt	$(H \text{ believe } (S \text{ believe } (S \text{ want } A)))$	$(H \text{ believe } (S \text{ believe } \varphi))$

Povšimněte si, že efektem *Inform* není „posluchač uvěří φ “, ale jen „posluchač uvěří, že mluvčí věří φ “. Autonomní agent není povinen věřit tomu, co mu kdo řekne — to je přesně ten rozdíl oproti volání metody.

4.7 KQML a FIPA-ACL

KQML — Knowledge Query and Manipulation Language

Vznikl v 90. letech v rámci projektu DARPA *Knowledge Sharing Effort* (KSE) společně s KIF.

- **KQML** = *vnější* komunikační jazyk („obálka dopisu“); nese *ilokuční* část zprávy.
- **KIF** = *vnitřní* jazyk; propoziční obsah zprávy, reprezentace znalostí.

Zpráva je seznam *performativa* a pojmenovaných parametrů:

```
(ask-one
  :content (PRICE IBM ?price)
  :receiver stock-server
  :language LPROLOG
  :ontology NYSE-TICKS)
```

Parametry: content, force, reply-with, in-reply-to, sender, receiver, language, ontology.

Performativa: achieve, advertise, ask-about, ask-one, ask-all, ask-if, break, sorry, error, broadcast, forward, recruit-one, recruit-all, reply, subscribe, tell, . . .

Kritika KQML: chyběl performativ *commit* (závazek), sémantika nebyla přesně definována, množina performativ byla nesystematická a zbytečně velká, transportní mechanismus nebyl standardizován.

FIPA-ACL — Agent Communication Language

FIPA (*Foundation for Intelligent Physical Agents*, dnes standardizační komise IEEE) navrhla ACL jako **zjednodušení a zpřesnění KQML**: menší a systematictější množina performativ a *formálně definovaná sémantika*. Praktická implementace: platforma JADE.

Struktura zprávy (13 polí — performativ a 12 parametrů; povinné je jen **performative**):

Pole	Význam
performative	typ komunikačního aktu (ilokuční síla) — <i>povinné</i>
sender, receiver, reply-to	identifikátory agentů (AID)
content	vlastní obsah zprávy
language	jazyk obsahu (SL, KIF, ...)
encoding, ontology	kódování a ontologie pro interpretaci obsahu
protocol	interakční protokol, jehož je zpráva součástí
conversation-id, reply-with, in-reply-to	párování zpráv do konverzace
reply-by	termín, do kdy se očekává odpověď

(inform
:sender agent1
:receiver agent2
:content (price good2 150)
:language sl
:ontology hpl-auction)

Performativa FIPA-ACL (22 celkem, nejdůležitější):

Skupina	Performativa
předávání informace	inform, inform-if, inform-ref, confirm, disconfirm
žádost o informaci	query-if, query-ref, subscribe
žádost o akci	request, request-when, request-whenever, cancel
vyjednávání	cfp (<i>call for proposals</i>), propose, accept-proposal, reject-proposal
provádění akcí	agree, refuse, failure
chyby, směrování	not-understood, proxy, propagate

♦ nad rámec slidů: formální sémantika SL ve slidech není, performativa ano]

Sémantika FIPA-ACL: FP a RE

Sémantika je definována v jazyce SL (*Semantic Language*) — modální logice s operátory $B_i\varphi$ (agent i věří φ), $U_i\varphi$ (je nejistý), $I_i\varphi$ (má záměr), Feasible, Done. Ke každému performativu jsou dány:

- **FP** (*feasibility precondition*) — co musí platit, aby odesílatel mohl akt provést;
- **RE** (*rational effect*) — efekt, kvůli němuž akt provádí. Příjemce *není povinen* RE naplnit (autonomie!) — RE je jen důvod odesílatele.

Například pro $\langle i, \text{inform}(j, \varphi) \rangle$:

$$\text{FP: } B_i\varphi \wedge \neg B_i(Bif_j\varphi \vee Uif_j\varphi), \quad \text{RE: } B_j\varphi.$$

Tedy: „já φ věřím a nevěřím, že už j o φ ví (že by věděl, zda platí, či neplatí — Bif — ani že by o tom měl mínění s nejistotou — Uif); chci, aby j uvěřil φ .“ Pro $\langle i, \text{request}(j, \alpha) \rangle$: FP: $FP(\alpha)[i \setminus j] \wedge B_i \text{Agent}(j, \alpha) \wedge \neg B_i I_j \text{Done}(\alpha)$, RE: $\text{Done}(\alpha)$.

Praktický problém: tato sémantika je neverifikovatelná — nelze zvenčí ověřit, že agent skutečně věří tomu, co tvrdí (*sincerity assumption*). To je známá slabina mentalistické sémantiky ACL; alternativou jsou *sociální* sémantiky založené na veřejných závazcích (*commitments*).

4.8 Interakční protokoly

Jednotlivá zpráva nestačí; agenti vedou **konverzace** s předepsanou strukturou. Interakční protokol (*interaction protocol*) je vzor přípustných posloupností zpráv; FIPA jich standardizuje kolem tuctu (specifikace FIPA-IP, notace AUML).

Protokol	Průběh
FIPA-Request	$I \rightarrow P$: request. $P \rightarrow I$: refuse (konec), nebo agree; po vykonání inform-done / inform-result, případně failure. Odpověď může být i not-understood.
FIPA-Query	$I \rightarrow P$: query-if / query-ref. P : refuse nebo agree, dále inform s výsledkem nebo failure.
FIPA-Contract-Net	viz níže: $\text{cfp} \rightarrow n \times (\text{propose} / \text{refuse}) \rightarrow \text{accept-proposal}$ vítězi + reject-proposal ostatním $\rightarrow \text{inform-done} / \text{failure}$.
FIPA-Iterated-Contract-Net	jako CNET, ale iniciátor může po přijetí nabídek vyhlásit <i>další kolo cfp</i> (vyjednávání o lepších podmínkách).
FIPA-Subscribe	I : subscribe; P posílá inform pokaždé, když se sledovaná hodnota změní, dokud nepřijde cancel.
FIPA-Brokering	/ I požádá zprostředkovatele (<i>broker</i>), ten najde vhodné poskytovatele, deleguje na ně požadavek a výsledky vrátí (příp. je nechá odpovědět přímo).
Recruiting	
Aukční protokoly	FIPA-English-Auction, FIPA-Dutch-Auction (viz odst. 5.11).

Poznámka k návrhu. Protokol určuje, jaké zprávy jsou v daném stavu konverzace legální — tím zavádí *předvídatelnost* do jinak autonomního chování a umožňuje implementovat agenta jako konečný automat nad stavy konverzace (přesně to dělá JADE třídami typu AchieveREInitiator, ContractNetResponder apod.).

4.9 Shrnutí ke zkoušce

- Ontologie = explicitní formalizovaný popis části reality (glosář + thesaurus); Gruberova definice. Formalismus: třídy, instance, vlastnosti, relace (zejména tranzitivní *is-a*), fakta. Ontologie + fakta = znalostní báze.
- Škála formalizace od slovníku po libovolná logická omezení; dělení na aplikační, doménové a horní ontologie.
- RDF = trojice *subjekt-predikát-objekt*, jednoduché a rychlé; RDFS přidává třídy.
- OWL = deskripční logika (rozhodnutelný fragment FOL), *open world assumption*, T-Box vs. A-Box; profily Lite/DL/Full (*SHIF*, *SHOIN*), OWL 2 = *SRIOQ* + profily EL/QL/RL.
- KIF = FOL v LISPovské syntaxi, obsah zprávy; KQML = obálka.
- Epistemická logika: $K_i\varphi$ + možné světy; znalost = S5 (axiom T), přesvědčení = KD45. Společná znalost $C_G\varphi$ nevznikne nespolehlivou komunikací (koordinovaný útok).
- Komunikace agentů je *akce*, ne volání metody — příjemce není povinen vyhovět.
- Austin: řečové akty; lokuce / ilokuce / perlokuce; Searle: podmínky zdařilosti a pět kategorií (asertiva, direktiva, komisiva, expresiva, deklarace); performativní sloveso + propoziční obsah.
- Cohen & Perrault: sémantika řečových aktů ve stylu STRIPS (pre / efekt) s modálními operátory; umět napsat Request(S,H,A) a Inform(S,H, φ) — efektem Inform je H věří, že S věří φ .
- KQML (obálka) + KIF (obsah); parametry a performativa; kritika (chybí commit, nejasná sémantika).
- FIPA-ACL: struktura zprávy (performative, sender/receiver, content, language, ontology, protocol, conversation-id, reply-by), 22 performativ, sémantika přes FP a RE v jazyce SL; problém neverifikovatelnosti.
- Protokoly: FIPA-Request, FIPA-Query, Contract Net (+ iterovaný), Subscribe, Brokering, aukční protokoly.

5 Distribuované řešení problémů, kooperace, teorie her a aukce

Nejobsáhlejší věta okruhu spojuje dvě různé situace, a kapitola proto má dvě poloviny. V první (odst. 5.1–5.6) agenti *sdílejí cíl* a jde jen o to, jak si rozdělit práci — to je *distribuované řešení problémů* a *kooperace*. Ve druhé (odst. 5.7–5.13) agenti sledují *vlastní zájmy*, takže je nutná teorie her (Nashova ekvilibria, Paretova efektivita) a mechanismy odolné vůči manipulaci (alokace zdrojů, aukce).

5.1 Koherence a koordinace

Agenti mají různé cíle, jsou autonomní, pracují v čase a rozhodují se za běhu; musí být schopni *dynamické koordinace*. Sdílejí přitom dvě věci: **úlohy** (*task sharing*) a **informace** (*result sharing*).

Koherence a koordinace

Koherence (*coherence*) — jak dobře systém funguje *jako celek* (kvalita řešení, efektivita využití zdrojů, degradace při poruchách).

Koordinace (*coordination*) — jak dobře se agentům daří *minimalizovat režii* spojenou se synchronizací: vyhnout se zbytečně duplicitní práci, konfliktům o zdroje a zbytečné komunikaci. Systém může být koherentní i při špatné koordinaci (agenti udělají práci třikrát, ale výsledek je správný) — za cenu plýtvání.

CDPS — Cooperative Distributed Problem Solving

Lesser a kol., 80. léta. Kooperace jednotlivých agentů při řešení problému, který přesahuje jejich individuální schopnosti (informace, zdroje, výpočetní kapacitu).

Klíčový předpoklad: agenti *implicitně sdílejí společný cíl* a nemají konflikty — **benevolence**. Měřítkem úspěchu je *výkon systému jako celku*; agent pomáhá celku, i když je to pro něj samotného nevýhodné. Benevolence *enormně zjednodušuje* návrh systému.

	Charakteristika	Rozdíl
PPS (<i>parallel problem solving</i>)	Bond, Gasser, 80. léta; důraz na paralelní řešení, homogenní a jednoduché procesory	agenti nejsou autonomní, jde v podstatě o paralelní algoritmus
CDPS	heterogenní agenti se sofistikovanými schopnostmi, sdílený cíl, benevolence	konflikty se neřeší, protože nevznikají
MAS	společnost agentů s <i>vlastními</i> cíli, které <i>nesdílejí</i>	je nutné řešit: proč a jak kooperovat, jak identifikovat a řešit konflikty, jak vyjednávat a smlouvat

5.2 Sdílení úloh a sdílení výsledků

Obecný postup CDPS má tři fáze:

1. **Dekompozice problému** — hierarchická, rekurzivní. Otázky: *jak* rozložit a *kdo* rozklad provede. Krajní varianta — model ACTORS: pro každý podproblém vznikne nový agent, až na úrovni jednotlivých instrukcí.
2. **Řešení podproblémů** — agenti si během něj typicky sdílejí informace a mohou potřebovat synchronizovat akce.
3. **Syntéza řešení** — opět hierarchická, skládání dílčích výsledků.

Sdílení úloh vyžaduje *dohodu* agentů (kdo co udělá) — typickým mechanismem je Contract Net. **Sdílení výsledků** může být *proaktivní* (agent pošle výsledek, o němž si myslí, že se bude hodit) nebo *reaktivní* (pošle ho na vyžádání).

Přínosy sdílení výsledků

- **Jistota** (*confidence*) — nezávislá řešení téhož problému lze porovnat; shoda zvyšuje důvěru ve výsledek.
- **Úplnost** (*completeness*) — agenti sdílejí své *lokální* pohledy a tím vzniká globálnější představa o problému.
- **Přesnost** (*precision*) — sdílení může zpřesnit celkové řešení.
- **Včasnost** (*timeliness*) — zjevná výhoda distribuce: zkrácení času.

5.3 Contract Net

Contract Net Protocol (CNET)

Smith a Davis, 1977–1980. Metafora **sdílení úloh pomocí smluv** podle vzoru trhu: agent v nouzi je *manažer* (*manager*), ostatní jsou *dodavatelé* (*contractors*). Čtyři fáze:

1. **Rozpoznání** (*recognition*) — agent zjistí, že má problém, který nezvládne sám (chybí mu schopnost, zdroje, informace, nebo by to bylo horší řešení).
2. **Ohlášení** (*announcement*) — rozešle (typicky broadcastem) oznámení úlohy: *popis úlohy* (může být i spustitelný), *omezení* (termín, cena, kvalita) a *meta-informace* (kdo se smí hlásit).
3. **Nabídky** (*bidding*) — příjemci se rozhodnou, zda mají zájem a schopnosti, a pošlou nabídku (*tender*) s cenou / časem / kvalitou.
4. **Přidělení a realizace** (*awarding, expediting*) — manažer vybere vítěze, uzavře s ním smlouvu; ten úlohu vykoná a vrátí výsledek.

Vlastnosti: jednoduchý, decentralizovaný, robustní vůči výpadkům; dodavatel se může sám stát manažerem podúlohy, což vede k *hierarchickým kaskádám subkontraktů*. Dynamicky rozděluje zátěž. Je nejčastěji implementovaným koordinačním mechanismem vůbec (v FIPA je standardizován jako FIPA-Contract-Net; v JADE jako hotové chování).

Limity: broadcast je drahý při mnoha agentech; předpokládá benevolenci (dodavatel nelže o svých schopnostech) — jinak je nutné použít aukční mechanismy odolné vůči manipulaci (odst. 5.11).

5.4 Systémy s tabulí (blackboard)

Blackboard system (BBS)

Vůbec první schéma pro kooperativní řešení problémů (HEARSAY-II, rozpoznávání řeči, 70. léta). Výsledky se sdílejí přes společnou datovou strukturu — **tabuli** (*blackboard*).

Metafora: kolem tabule sedí několik expertů, kteří na ni mohou psát i z ní číst. Na tabuli se dynamicky objevují úlohy; jakmile některý expert vidí úlohu, kterou umí řešit, zapíše na tabuli částečné řešení. Takto se pokračuje, dokud se na tabuli neobjeví řešení celé úlohy.

Role:

- **Tabule** — sdílená paměť; typicky strukturovaná do několika *úrovní abstrakce* (hierarchie cílů a akcí). Zásadní je formalismus reprezentace informace.
- **Experti** (*knowledge sources*) — agenti řešící konkrétní typ úlohy; reagují na cíle na tabuli, svůj cíl přebírají z tabule a po vybrání vykonají akci. Mají seznam schopností a efektivitu.
- **Arbitr** (*arbiter, control component*) — vybírá, který expert smí k tabuli. Může být čistě reaktivní, nebo může uvažovat plány a maximalizovat očekávanou utilitu. Odpovídá za řešení problému na vyšší úrovni.

Nutnost vzájemného vyloučení nad tabulí — úzké hrdlo systému.

Klady a zápory BBS:

- + Jednoduchý mechanismus koordinace, kooperace, sdílení úloh i výsledků.
- + Experti o sobě *nemusí vědět* a přesto spolupracují (*loose coupling*); přidání experta nevyžaduje změnu ostatních.

- + Zprávy na tabuli lze přepisovat — delegovat úlohy, vytvářet podúlohy, měnit experty.
- Všichni agenti musí mít přístup k tabuli a sdílet stejnou architekturu; kolem tabule bývá „narváno“ (částečně řeší distribuované hašovací tabulky).

Příklad: BBWar. Tabule je hašovací tabulka mapující požadované schopnosti na úlohy; „otevřené mise“ jsou publikovány na tabuli. Experti tvoří hierarchii: agent *velitel* hledá úlohy typu „dobyj město“ (ATTACK-CITY) a převádí je na několik úloh „zaútoč na pozici“ (ATTACK-LOCATION); vojáci různých typů si pak z tabule vybírají mise, na které mají schopnosti.

Příklad: FELINE (Wooldridge, Jennings, 1990) — distribuovaný expertní systém z doby před KQML a ACL. Sdílení znalostí a distribuce podúloh; každý agent je pravidlový systém, který si udržuje: *Skills* — „umím dokázat/vyvrátit následující. . .“ a *Interests* — „zajímá mě, zda následující platí. . .“. Komunikace je trojice (odesílatel, příjemce, obsah), kde obsah je hypotéza + řečový akt (request, response, inform).

5.5 Distribuované omezující podmínky (DisCSP a DCOP)

♦ nad rámec slidů: není ve slidech; okruh ale mluví o „distribuovaném řešení problémů“, kam formální větve patří

DisCSP a DCOP

Distribuovaný CSP: proměnné $x_1, \dots, x_n$ s doménami D_i jsou rozděleny mezi agenty (typicky jedna proměnná na agenta) a omezení propojují proměnné *různých* agentů. Hledá se přiřazení splňující všechna omezení; žádný agent nevidí celý problém a komunikuje jen se sousedy v grafu omezení.

DCOP je optimalizační varianta: omezení jsou *měkká*, zadaná funkcemi ceny $f_{ij}(x_i, x_j)$, a maximalizuje se $\sum_{ij} f_{ij}$ — tedy sociální blahobyt.

Algoritmy: *ABT* (asynchronní backtracking, Yokoo) — agenti jsou uspořádáni a posílají si zprávy ok? (své přiřazení níž) a nogood (zpět nahoru při konfliktu); dále *ADOPT*, *DPOP* a *Max-Sum*. Aplikace: rozvrhování schůzek, přidělování úkolů senzorům, alokace frekvencí.

5.6 Interakce agentů v prostředí

Agenti si mohou navzájem pomáhat i překážet, a to i *bez komunikace*, prostě tím, že ovlivňují prostředí:

- Roboti mohou přenést cihlu, jen když tlačí ze dvou stran současně (pozitivní interakce, vyžaduje synchronizaci).
- Roboti se natlačí ke dveřím a nemohou je otevřít (negativní interakce, vyžaduje koordinaci).

Agenti mohou vytvářet společenství, hierarchie i nepřátelství. **Je nutné znát typy interakcí** v konkrétním MAS — jinak nelze navrhnout efektivní řídicí mechanismy. Nejjednodušší modelový případ, kterým se zabývá následující kapitola, je interakce dvou racionálních (sobeckých) agentů v prostředí připomínajícím hru.

5.7 Sobecký agent

Proč by měl být agent upřímný ohledně svých schopností? Proč by měl dokončit přidělenou úlohu? Je-li systém homogenní (celý navržený námi), je benevolence dobrá strategie. Většinou ale systém obsahuje agenty s různými zájmy — konflikt mezi společným cílem a cíli jednotlivců (např. řízení letového provozu: všichni chtějí bezpečnost, ale každá aerolinka chce přistát první). Někdy ani není zřejmé, co je společný zájem. Pak je lépe uvažovat **sobecké** (*self-interested*) agenty.

Sobecký agent

Množina možných výsledků $O = \{o_1, o_2, \dots\}$ je *společná* pro všechny agenty, ale preference nikoli: každý agent i má vlastní **utilitní funkci** $u_i : O \rightarrow \mathbb{R}$, která výsledky uspořádá.

Strategické myšlení: maximalizuj svou utilitu za předpokladu, že všichni ostatní jednají rovněž racionálně. Toto *nemaximalizuje* společnou utilitu, ale je to *robustní* strategie.

Poznámka: peníze nejsou dobrou utilitou pro člověka — utilitní funkce peněz je nelineární a liší se pro bohaté a chudé.

5.8 Hra v normální formě

Hra dvou hráčů v normální formě

Agenti i a j s množinami akcí A_i, A_j . Výsledek je dán oběma: $e : A_i \times A_j \rightarrow O$. Agent volí **strategii** s_i :

- **ryzí** (*pure*) — deterministická volba jedné akce;
- **smíšená** (*mixed*) — rozdělení pravděpodobnosti nad ryzími strategiemi (náhodný výběr).

Hru zapisujeme **výplatní maticí** (*payoff matrix*): řádky = strategie hráče i , sloupce = strategie hráče j , buňka obsahuje dvojici u_i/u_j .

Obecně: hra v normální formě je trojice $\langle N, (A_i)_{i \in N}, (u_i)_{i \in N} \rangle$, kde N je množina hráčů.

Dominance

Strategie s_i **silně dominuje** strategii s'_i , pokud dává agentovi i *striktně* lepší výsledek proti *všem* strategiím protihráče; **slabě dominuje**, pokud dává výsledek alespoň stejně dobrý proti všem a striktně lepší alespoň proti jedné (tak dominanci definuje i přednáška). Strategie je **dominantní**, pokud dominuje všem ostatním strategiím agenta i .

Rozdíl není akademický: pravdomluvnost ve Vickreyově aukci je jen *slabě* dominantní (odst. 5.11).

Racionální agent hraje dominantní strategii, existuje-li. *Iterované odstraňování dominovaných strategií* (IESDS) je základní řešící technika: dominovanou strategii nikdy nikdo nezahraje, lze ji tedy z matice vyškrtnout, což může učinit dominovanými další strategie. Při odstraňování *silně* dominovaných strategií nezáleží na pořadí a výsledek je jednoznačný; u *slabě* dominovaných strategií na pořadí **záleží** a lze přijít o některá ekvilibria.

Nashovo ekvilibrium

Dvojice strategií (s_1^*, s_2^*) je v **Nashově ekvilibriu**, pokud:

- hraje-li agent i strategii s_1^* , je pro agenta j nejlepší hrát s_2^* , a zároveň
- hraje-li agent j strategii s_2^* , je pro agenta i nejlepší hrát s_1^* .

Tedy s_1^* a s_2^* jsou *vzájemně nejlepší odpovědi* (*mutual best responses*). Obecně pro n hráčů: profil $s^* = (s_1^*, \dots, s_n^*)$ je NE, pokud pro každého i a každou jeho strategii s_i platí

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*),$$

kde s_{-i} značí strategie všech ostatních. **Žádný hráč nemá jednostranný důvod se odchýlit.** Naivní hledání ekvilibrií pro n agentů a m strategií vyžaduje projít m^n profilů. Tato definice se týká *ryzích* strategií — ne každá hra má NE v ryzích strategiích (kámen–nůžky–papír) a některé hry jich mají více.

Věta 5.1 (Nashova věta, 1950). *Každá hra s konečným počtem hráčů a konečným počtem strategií má alespoň jedno Nashovo ekvilibrium ve smíšených strategiích.*

Výpočetní složitost. Hledání NE je *totální* vyhledávací problém (řešení vždy existuje, jde jen o to ho najít). Od roku 2006 víme, že je **PPAD-úplný** — velmi zjednodušeně řečeno „o něco méně beznadějně než NP-úplnost“ (NP-úplnost pro problém, který má vždy řešení, ani nedává smysl). Pro tři a více hráčů to dokázali Daskalakis, Goldberg a Papadimitriou; těsně poté Chen a Deng doplnili

i zbývající a nejtěžší případ *dvou* hráčů.

Paretoova efektivita a sociální blahobyt

Profil strategií je **Pareto-optimální** (*Pareto efficient*), pokud neexistuje jiný profil, který by zlepšil výsledek jednoho agenta, aniž by zhoršil výsledek některého jiného. Ne-Pareto-optimální řešení lze tedy zlepšit „zadarmo“.

Sociální blahobyt (*social welfare*) profilu je součet utilit všech agentů: $sw(o) = \sum_{i \in N} u_i(o)$. Maximalizace sociálního blahobytu dává smysl v kooperativních scénářích — když jsou agenti z jednoho týmu, mají jednoho vlastníka, řeší jednu úlohu; čím homogennější systém, tím lépe. Maximum sociálního blahobytu je vždy Pareto-optimální, opačně to neplatí.

5.9 Klasické hry

Věžňovo dilema (*Prisoner's dilemma*)

$i \backslash j$	C (spolupráce)	D (zrada)
C	3/3	0/4
D	4/0	1/1

C = *cooperate*, spolupracovat se spolupachatelem; D = *defect*, udat ho a dostat nižší trest. Kanonické značení výplat: R (*reward* za oboustrannou spolupráci), P (*punishment* za oboustrannou zradu), T (*temptation* — zradím spolupracujícího), S (*sucker's payoff* — jsem zrazen). Podmínky věžňova dilematu:

$$T > R > P > S \quad \text{a často navíc} \quad 2R > T + S.$$

- $R > P$: oboustranná spolupráce je lepší než oboustranná zrada.
- $T > R$ a $P > S$: *zrada je dominantní strategie pro oba agenty*.
- (D, D) je **jediné Nashovo ekvilibrium**.
- (D, D) je zároveň **jediné ne-Pareto-optimální** řešení hry.
- (C, C) maximalizuje sociální blahobyt ($3 + 3 = 6$).

Racionalita každého jednotlivce tedy vede ke kolektivně nejhoršímu výsledku.

Důsledky věžňova dilematu:

- **Tragédie obecní pastviny** (*tragedy of the commons*): sdílený zdroj (pastvina, rybolov, atmosféra) je vyčerpán, protože každý jednotlivec maximalizuje svůj užitek.
- Otázka, *co vlastně znamená být racionální* — a zda jsou lidé racionální (experimentálně lidé spolupracují častěji, než teorie předpovídá).
- **Stín budoucnosti** (*shadow of the future*): v *iterovaném* věžňově dilematu se situace mění.

Iterované věžňovo dilema a Axelrodovy turnaje

Hraje-li se PD *konečněkrát* a *oba to vědí*, zpětnou indukcí vyjde D v každém kole (v posledním kole není důvod spolupracovat, tedy ani v předposledním, ...). Hraje-li se *nekonečně* (nebo s neznámým koncem, tj. s pravděpodobností pokračování), stává se spolupráce racionální — *folk theorem*: každý výplatní profil lepší než minimax lze podepřít jako ekvilibrium opakované hry. Robert Axelrod uspořádal v roce 1980 turnaje strategií pro iterované PD. Vyhrála (dvakrát) nejjednodušší přihlášená strategie **Tit For Tat** (TFT, „půjčka za oplátku“): *v prvním kole spolupracuj, pak vždy zopakuj poslední tah soupeře*. Axelrod z výsledků odvodil čtyři vlastnosti úspěšných strategií:

- **slušnost** (*nice*) — nezrazuj jako první;
- **odvetnost** (*retaliating*) — na zradu okamžitě odpověz;
- **odpouštění** (*forgiving*) — po návratu soupeře ke spolupráci mu odpust;
- **nezávistivost** (*non-envious*) — neusiluj o to porazit *konkrétního* soupeře; TFT nikdy nezís-

kal víc bodů než jeho protihráč, a přesto vyhrál turnaj. Jako pátou vlastnost Axelrod uvádí **srozumitelnost** (*clarity*): strategie musí být pro protihráče čitelná, jinak s ní nelze navázat spolupráci. Slabina TFT: při šumu (chybně provedený tah) se dva TFT hráči zaseknou ve vzájemné odvetě — proto varianty *Generous TFT* (občas odpustí) a *Tit for Two Tats*.

Koordinální hra

$i \backslash j$	Balet	Zápas
Balet	1/2	0/0
Zápas	0/0	2/1

Známa též jako *Battle of the Sexes* (BoS) nebo *Bach or Stravinsky*. Výplaty podporují shodu strategií, hráči se ale liší v tom, na které shodě jim záleží víc.

- Nashova ekvilibria v ryzech strategiích: (Balet, Balet) a (Zápas, Zápas).
- Sociální blahobyť i Paretova optimalita: opět obě tyto dvojice.
- **Smíšené NE**: každý hráč hraje svou *preferovanější* možnost s pravděpodobností $2/3$ a méně preferovanou s $1/3$. Očekávaná utilita ve smíšeném ekvilibriu je jen $2/3$ — méně než v kterémkoli ryzím ekvilibriu.

Výpočet smíšeného NE (obecný recept pro hry 2×2): hráč j volí pravděpodobnost q hraní Baletu tak, aby byl hráč i *indiferentní* mezi svými akcemi:

$$u_i(\text{Balet}) = 1 \cdot q + 0 \cdot (1 - q), \quad u_i(\text{Zápas}) = 0 \cdot q + 2 \cdot (1 - q).$$

Rovnost $q = 2 - 2q$ dává $q = 2/3$, tj. hráč j hraje Balet (svou preferovanou volbu) s pravděpodobností $2/3$. Symetricky hráč i hraje Zápas s pravděpodobností $2/3$. Očekávaná utilita hráče i je pak $1 \cdot \frac{2}{3} = \frac{2}{3}$. **Klíčové pravidlo: ve smíšeném NE volí každý hráč pravděpodobnosti tak, aby byl protihráč indiferentní.**

Antikoordinální hra (*Chicken, Hawk–Dove*). Výplaty naopak podporují volbu *různých* strategií: $u(\text{uhnu, uhnu}) = 5/5$, $u(\text{uhnu, nedám se}) = 1/6$, $u(\text{nedám se, uhnu}) = 6/1$, $u(\text{nedám se, nedám se}) = 0/0$. Ryzí NE jsou obě *asymetrické* dvojice; maximum sociálního blahobytu (oba uhnou, $5+5$) naopak *není* NE. Symetrické smíšené NE je $p = 1/2$ s očekávanou utilitou 3.

5.10 Sociální volba a volby

♦ nad rámec slidů: ve slidech je celá kapitola, oficiální znění okruhu ji ale nejmenuje — zařazeno jako doplněk k agregaci preferencí

Teorie her řeší, jak se agenti zachovají při daných preferencích. *Teorie sociální volby* se ptá opačně: jak z individuálních preferencí *odvodit* kolektivní rozhodnutí. **Funkce sociálního blahobytu** agreguje preference do úplného uspořádání $>_{sw}$, **funkce sociální volby** vybírá jednoho vítěze.

Volební systémy: *většinový* (nad 50 %); *prostá většina* (*plurality* — bod za každé první místo); *prostá většina s vyřazováním* (IRV — opakovaně vyřadí nejslabšího a přerozděl jeho hlasy); *sekvenční většinové volby* (pavouk párových soubojů — pořadí zápasů ovlivňuje výsledek); *Bordova metoda* ($N - 1$ bodů za první místo, ..., 0 za poslední — konsenzuální, nikoli většinové hlasování); *Slaterův systém* (acyklické uspořádání minimalizující počet obrácených hran v turnajovém grafu — NP-těžké).

Bordovy kombinace opravují hlavní slabinu Bordovy metody — to, že nemusí zvolit Condorcetova vítěze. Všechny tři ho zvolí, pokud existuje: *Blackova metoda* (nejdřív zkus Condorcetovu metodu; není-li vítěz, vezmi Bordova); *Baldwinova metoda* (spočti Bordova skóre, vyřadí kandidáta s nejnižším, skóre přepočti jen mezi zbylými a opakuj); *Nansonova metoda* (totéž, ale v každém kole se vyřadí všichni pod průměrem — rychlejší konvergence).

Vítěz prosté většiny nemusí být většinový: při $o_1 = 40\%$, $o_2 = 30\%$, $o_3 = 30\%$ vyhrává o_1 , ačkoli

ho 60 % voličů nechtělo. Odtud **taktické hlasování**.

Condorcetův paradox

Existují situace, kdy *ať zvolíme kterýkoli výsledek, většina voličů s ním bude nespokojena*.

Příklad: $V_1 : A > B > C$, $V_2 : B > C > A$, $V_3 : C > A > B$. Párové souboje: A porazí B (2:1), B porazí C (2:1), C porazí A (2:1). **Kolektivní preference je cyklická**, přestože všechny individuální preference jsou lineární — neexistuje *Condorcetův vítěz*. Motiv je stejný jako u neexistence ryzího Nashova ekvilibria v kámen–nůžky–papír.

Kritéria volebních systémů

Paretova podmínka (jednomyslnost): preferují-li X před Y *všichni* voliči, má platit $X >_{sw} Y$. *Splňuje* majority a Borda, *nesplňuje* sekvenční většinové.

Podmínka Condorcetova vítěze: kdo porazí všechny v párovém srovnání, má vyhrát. *Splňuje* sekvenční většinové volby, *nesplňuje* plurality ani Borda.

Nezávislost na irelevantních alternativách (IIA): zda $X >_{sw} Y$, má záviset pouze na relativním pořadí X a Y u voličů. *Nesplňuje* prakticky žádný běžný systém.

Neomezený definiční obor (univerzalita) a **nediktátorství**.

Věta 5.2 (Arrowova věta o nemožnosti, 1951). *Pro tři a více kandidátů neexistuje preferenční volební systém, který by převedl individuální uspořádání na úplné a tranzitivní společenské uspořádání a zároveň splňoval neomezený definiční obor, nediktátorství, Paretovu podmínku a IIA.*

Jednodušší formulace: pro více než dva kandidáty je jediná procedura splňující Paretovu podmínku a IIA *diktatura*. Jde o *negativní* výsledek o limitech kolektivního rozhodování — neplyne z něj, že jediným funkčním systémem je diktatura, nýbrž že každý reálný systém musí některé kritérium obětovat (nejčastěji IIA).

Gibbardova–Satterthwaiteova věta: každá surjektivní a nediktátorská funkce sociální volby pro tři a více kandidátů je **manipulovatelná**. Pro MAS je to zásadní: agent je počítač a taktické hlasování mu nedělá morální problém — proto se hledají mechanismy odolné vůči manipulaci, což je téma aukcí.

5.11 Aukce jako mechanismus alokace zdrojů

Aukce

Aukce je **tržní instituce**, v níž zprávy od obchodníků obsahují cenovou informaci — nabídku ke koupi za danou cenu (*bid*), nebo nabídku k prodeji za danou cenu (*ask*) — a která dává přednost vyšším nabídkám ke koupi a nižším nabídkám k prodeji.

V MAS je aukce především **mechanismus, jak rozdělit vzácné zdroje mezi agenty**. *Reálně*: eBay, Sotheby's, těžební povolení, frekvenční pásma pro mobilní operátory. *V informatice*: přidělení procesorového času, šířky pásma, výpočetních úloh, reklamních pozic.

Efektivita aukce: přidělí zdroj agentovi, který ho chce nejvíce (má nejvyšší ocenění). Prodávající chce *maximalizovat* cenu, kupující *minimalizovat*; každý kupující má vlastní utilitní funkci.

Klasifikace aukcí

Podle ocenění předmětu:

- *soukromá hodnota* (*private value*) — hodnota závisí jen na kupujícím (lístek na koncert);
- *společná hodnota* (*common value*) — hodnota je pro všechny stejná, jen ji neznají (těžební práva) — odtud *prokletí vítěze* (*winner's curse*): vyhrává ten, kdo se nejvíce spletl směrem nahoru;
- *korelovaná hodnota* — kombinace.

Podle protokolu: vítěz platí první / druhou cenu; otevřené vyvolávání (*open cry*) vs. obálková metoda (*sealed bid*); jedno kolo vs. více kol.

5.12 Čtyři základní typy aukcí

Aukce	Protokol	Racionální strategie	Poznámky
Anglická	otevřené vyvolávání, rostoucí cena, platí se první (nejvyšší) cena	<i>dominantní</i> : přihazovat po ε , dokud cena nedosáhne mého ocenění, pak odejít	Nejběžnější (starožitnosti, umění, internet — asi 95 % aukcí). Typický případ, kdy kupující cenu přeplatí.
Holandská	otevřené vyvolávání, klesající cena, první, kdo přijme, platí aktuální cenu	dominantní strategie <i>neexistuje</i> ; čekat, až cena klesne těsně pod mé ocenění — ale je to hazard	Rychlá; zboží podléhající zkáze (květiny, ryby). Oblíbená u prodávajících. Neumožňuje odhadnout tržní cenu ani preference ostatních.
FPSB (<i>first-price sealed bid</i>)	jedno kolo, uzavřené obálky, vyhrává nejvyšší nabídka a platí svou cenu	dominantní strategie <i>neexistuje</i> ; nabídnout méně než své ocenění (<i>bid shading</i>) — optimum závisí na odhadu ostatních	Prodej nemovitostí, státních dluhopisů. Nenutí kupující použít svou utilitní funkci, neodhalí tržní cenu. <i>Strategicky ekvivalentní holandské aukci.</i>
Vickreyova (<i>second-price sealed bid</i>)	jedno kolo, uzavřené obálky, vyhrává nejvyšší nabídka, ale platí druhou nejvyšší	<i>dominantní</i> : nabídnout pravdivě své ocenění	Teoreticky nejoblíbenější; Google AdWords, filatelistické aukce. <i>Ekvivalentní anglické aukci (se soukromými hodnotami).</i>

Proč je pravdomluvnost dominantní strategií ve Vickreyově aukci

Nechť moje ocenění je v , nabídnou b a nejvyšší nabídka ostatních je p (tu neznám). Vyhraji, právě když $b > p$, a pak platím p ; můj zisk je $v - p$.

- **Nabídnou $b > v$ (nadhodnotím):** oproti $b = v$ se něco změní jen když $v < p < b$. Pak vyhraji, ale platím $p > v$ — zisk $v - p < 0$. *Škodí mi.*
- **Nabídnou $b < v$ (podhodnotím):** oproti $b = v$ se něco změní jen když $b < p < v$. Pak prohraji, ačkoli bych vyhrál se ziskem $v - p > 0$. *Přijdu o zisk.*
- V žádném případě *nemohu* nabídkou $b \neq v$ získat oproti $b = v$ víc.

Tedy $b = v$ je (slabě) dominantní strategie — bez ohledu na to, co dělají ostatní. Aukce je **odolná vůči manipulaci** (*strategy-proof, incentive compatible*) a zároveň **efektivní** (vyhrává ten s nejvyšším oceněním).

Příklad — srovnání výnosů. Tři kupující s oceněními $A = 80$ Kč, $B = 60$ Kč, $C = 30$ Kč. V ideálním scénáři (bez dodatečné informace, bez podvádění) dopadnou aukce takto:

Aukce	Vítěz	Cena
Anglická	A	≈ 61 (přihazuje se, dokud neodstoupí B na 60)
Holandská	A	80 nebo o něco méně (79) — A nesmí riskovat, že přijme až pozdě
FPSB	A	80 (resp. $60 + \varepsilon$, zná-li A ocenění ostatních)
Vickreyova	A	60 (druhá nejvyšší nabídka)

Ve všech případech získá předmět agent s nejvyšším oceněním (aukce jsou *efektivní*), liší se ale cena a informace, kterou aukce odhalí.

Proč u FPSB vychází 80, když se má nabízet pod svým oceněním? Protože scénář předpokládá *nulovou* informaci o ostatních: agent A neví, jak vysoko půjde B , a jakýkoli ústup od 80 riskuje prohru. Míra podhodnocení je funkcí přesvědčení o ostatních — proto FPSB (a holandská aukce) **nemá dominantní strategii**. Zná-li A ocenění ostatních, nabídne $60 + \varepsilon$ a výnos prodávajícího spadne na úroveň Vickreyovy aukce (srov. větu o ekvivalenci výnosů níže). U anglické a Vickreyovy aukce naproti tomu dominantní strategie existuje a nezávisí na tom, co dělají ostatní.

Věta o ekvivalenci výnosů (*revenue equivalence theorem*)

Vickrey (1961), Myerson, Riley–Samuelson. Za předpokladů: rizikově neutrální kupující, nezávislé soukromé hodnoty ze stejného spojitého rozdělení, předmět získá kupující s nejvyšším oceněním a kupující s nejnižším možným oceněním má nulový očekávaný zisk — **všechny čtyři základní aukce dávají prodávajícímu stejný očekávaný výnos.**

Rozdíly v praxi tedy pramení z porušení předpokladů: averze k riziku (pak jsou lepší FPSB/holandská), společné hodnoty a prokletí vítěze (pak je lepší anglická, protože odhaluje informaci), možnost koluze mezi kupujícími (anglická je zranitelnější), náklady na komunikaci (sealed bid je levnější).

5.13 Kombinatorické aukce a VCG

Kombinatorická aukce

Draží se *více předmětů současně* a kupující podávají nabídky na *podmnožiny (bundles)*. Motivace: hodnoty předmětů nejsou aditivní — *komplementarita* (levá a pravá bota, dva sousední frekvenční bloky: $v(\{x, y\}) > v(x) + v(y)$) a *substituce* (dva letenky na tentýž let: $v(\{x, y\}) < v(x) + v(y)$).

Problém určení vítězů (*winner determination problem*, WDP): vybrat takovou množinu vzájemně disjunktních nabídek, která maximalizuje součet cen. Jde o zobecnění množinového pakování — **NP-těžký** problém (a dokonce těžko aproximovatelný). Řeší se ILP solvery, větvením a mezemi (CABOB, CPLEX).

Druhý problém je *reprezentační*: nabídek na podmnožiny je $2^m - 1$; používají se kompaktní jazyky (OR, XOR, OR-of-XOR).

5.14 Další typy aukcí a alokace zdrojů

- *All-pay auction* — platí všichni, vyhrává nejvyšší nabídka; model lobbingu, úplatků, sportovních klání a patentových závodů.
- *Tichá* — písemná varianta anglické. *Amsterodamská* — začne jako anglická, zbudou-li dva zájemci, přepne se na holandskou s dvojnásobnou cenou. *Tsukiji* (tokijský rybí trh) — všichni nabídnou najednou, remízy se řeší kámen–nůžky–papír.
- *Dvojitá aukce* — nabídky zadávají kupující i prodávající a párují se; model burzy.

5.15 Návrh mechanismů, VCG a vyjednávání

♦ nad rámec slidů: ve slidech není; okruh jmenuje „alokaci zdrojů“, k níž toto patří — kombinatorické aukce slidy zmiňují jednou větou]

Návrh mechanismů (*mechanism design*)

Teorie her se ptá, *jak se agenti zachovají při daných pravidlech*; návrh mechanismů je „obrácená teorie her“: *jaká pravidla zvolit, aby racionální sobečtí agenti sami od sebe dělali to, co chceme* — typicky mluvili pravdu.

Mechanismus je dvojice: pravidlo výběru výsledku $x(\hat{v})$ z ohlášených preferencí a platební pravidlo $p_i(\hat{v})$; užitek je $u_i = v_i(x(\hat{v})) - p_i(\hat{v})$. Žádoucí vlastnosti: **kompatibilita s motivací** (vyplatí se ohlásit pravdu; nejsilnější varianta *strategy-proof* — pravdomluvnost je dominantní strategie), **individuální racionalita** ($u_i \geq 0$), **efektivita** (maximalizace sociálního blahobytu), **vyrovnaný rozpočet** a **výpočetní zvládnutelnost**. Vše najednou mít nelze.

Princip odhalení: ke každému mechanismu existuje *přímý* mechanismus, v němž agenti pravdivě ohlásí preference a výsledek je stejný. Proto se teorie točí kolem Vickreyho a VCG. Bez plateb naráží na Gibbardovu–Satterthwaiteovu větu.

Kombinatorické aukce a VCG

V **kombinatorické aukci** se draží více předmětů současně a nabídky se podávají na *podmnožiny*, protože hodnoty nejsou aditivní: *komplementarita* ($v(\{x, y\}) > v(x) + v(y)$, dva sousední frekvenční bloky) a *substitute* ($v(\{x, y\}) < v(x) + v(y)$). **Problém určení vítězů** (WDP) — vybrat disjunktí nabídky maximalizující součet cen — je **NP-těžký**.

VCG (Vickrey–Clarke–Groves) zobecňuje Vickreyovu aukci: zvolí se alokace x^* maximalizující $\sum_i \hat{v}_i(x)$ a agent i platí **externalitu, kterou způsobuje ostatním**:

$$p_i = \max_x \sum_{j \neq i} \hat{v}_j(x) - \sum_{j \neq i} \hat{v}_j(x^*).$$

Je pravdomluvný, efektivní a individuálně racionální; pro jeden předmět se redukuje na Vickreyovu aukci. Nevýhody: nutnost řešit WDP $n+1$ -krát, malý (i nulový) výnos prodávajícího a zranitelnost vůči koluzi.

Příklad: předměty $\{x, y\}$; agent 1: $\{x\} \rightarrow 5$, agent 2: $\{y\} \rightarrow 4$, agent 3: $\{x, y\} \rightarrow 8$. Optimální alokace je $5 + 4 = 9 > 8$. Platba $p_1 = 8 - 4 = 4$ (bez agenta 1 by ostatní měli 8, se zvolenou alokací mají 4), zisk $5 - 4 = 1$; symetricky $p_2 = 8 - 5 = 3$. Výnos prodávajícího $7 < 8$.

Vyjednávání: monotónní protokol ústupků a Zeuthenova strategie

Aukce vyžaduje centrálního aukcionáře; alternativou je přímé **vyjednávání**. *Vyjednávací množina* jsou individuálně racionální a Pareto-optimální dohody.

Monotónní protokol ústupků: v každém kole oba agenti současně navrhnou dohodu. Skončí, je-li návrh jednoho pro druhého alespoň tak dobrý jako jeho vlastní; jinak musí alespoň jeden *ustoupit*, jinak vyjednávání ztroskotá.

Zeuthenova strategie odpovídá, *kdo* má ustoupit — ten, kdo má menší ochotu riskovat konflikt:

$$\text{Risk}_i = \frac{u_i(\text{vlastní návrh}) - u_i(\text{návrh protějšku})}{u_i(\text{vlastní návrh})}.$$

Ustupuje agent s *nižším* Risk, a to právě tak málo, aby se poměry obrátily. Dvojice Zeuthenových strategií je v Nashově ekvilibriu a dohoda maximalizuje součin utilit (Nashovo vyjednávací řešení).

5.16 Shrnutí ke zkoušce

- Koherence (výkon celku) vs. koordinace (minimalizace synchronizační režie).
- CDPS: benevolence + sdílený cíl; odlišit od PPS (homogenní procesory, paralelní algoritmus) a od MAS (vlastní cíle, konflikty, vyjednávání).
- Tři fáze CDPS: dekompozice — řešení podproblémů — syntéza. Sdílení úloh (dohoda) vs. sdílení výsledků (proaktivní/reaktivní) a jeho čtyři přínosy: jistota, úplnost, přesnost, včasnost.
- **CNET**: recognition — announcement — bidding — awarding/expediting; manažer a dodavatel, hierarchické subkontrakty; standardizován jako FIPA-Contract-Net.
- **BBS**: tabule (hierarchie abstrakce, mutex), experti (skills), arbitr (výběr experta); volná vazba mezi experty; úzké hrdlo přístupu k tabuli.
- DisCSP / DCOP: proměnné a omezení rozdělené mezi agenty, nikdo nevidí celý problém; ABT (ok? / nogood); DCOP maximalizuje sociální blahobyt.
- Sobecký agent maximalizuje *svou* očekávanou utilitu za předpokladu, že ostatní dělají totéž; společná utilita se tím nemaximalizuje.
- Hra v normální formě, ryzí vs. smíšené strategie, dominance a dominantní strategie, iterované odstraňování dominovaných strategií.
- **Nashovo ekvilíbrio** = vzájemně nejlepší odpovědi, nikdo nemá důvod se jednostranně odchýlit; naivní hledání m^n ; **Nashova věta** (existence ve smíšených strategiích); hledání je PPAD-úplné.

- **Paretova optimalita** (nelze zlepšit jednomu, aniž by se jinému přitížilo) vs. **sociální blahobyť** (součet utilit); maximum sw je Pareto-optimální, obráceně ne.
- **Věžňovo dilemma**: $T > R > P > S$; D dominantní; (D, D) jediné NE a jediné ne-Pareto-optimální; (C, C) maximalizuje sw. Tragedy of the commons. Iterované PD, stín budoucnosti, Axelrod a TFT (slušná, odvetná, odpouštějící, nezávistivá, srozumitelná).
- Umět spočítat smíšené NE pro 2×2 : pravděpodobnosti volím tak, aby byl *protihráč* indiferentní.
- Sociální volba: plurality / IRV / sekvenční / Borda / Slater; Condorcetův paradox (cyklická kolektivní preference); kritéria Pareto, Condorcetův vítěz, IIA, univerzalita, nediktátorství; **Arrowova věta** (Pareto + IIA $\Rightarrow$ diktatura); Gibbard–Satterthwaite (každý systém lze manipulovat).
- Aukce = mechanismus alokace vzácných zdrojů; efektivní aukce dá zdroj tomu, kdo ho oceňuje nejvíc. Dimenze: private/common value, open cry/sealed bid, first/second price, počet kol.
- Čtyři základní typy a jejich strategie: anglická (přihazuj po ε do svého ocenění) a Vickrey (nabídni pravdivě) mají *dominantní* strategii; holandská (čekej na ocenění $-\varepsilon$) a FPSB (nabídni pod svým oceněním) *dominantní* strategii **nemají**. Umět **důkaz pravdomluvnosti** Vickreyho.
- Ekvivalence: anglická $\leftrightarrow$ Vickreyova, holandská $\leftrightarrow$ FPSB. **Věta o ekvivalenci výnosů** a kdy neplatí (averze k riziku, common values, prokletí vítěze, koluze). Umět příklad s oceněními 80/60/30.
- Návrh mechanismů (obrácená teorie her), princip odhalení; kombinatorické aukce a WDP (NP-těžký); **VCG** = alokace maximalizující sw + platba rovná externalitě.
- Vyjednávání: monotónní protokol ústupků + Zeuthenova strategie (ustupuje ten s nižším rizikem).

6 Metody pro učení agentů a zpětnovazební učení

♦ **nad rámec slidů: celá kapitola** Přednáška uvádí učení agentů explicitně mezi tématy, která *nepokrývá*. Okruh ho ale jmenuje, takže je nutné ho umět; kapitola je proto zpracována v rozsahu, který na zkoušce obstojí — přehled metod učení, MDP a Q-learning podrobně, zbytek jen rámcově.

6.1 Proč a jak se agenti učí

Inteligentní agenti v MAS by měli být *adaptivní*. Učení je změna chování agenta na základě zkušenosti; metod je celá škála:

Metody pro učení agentů

- **Podle učebního signálu** (tři klasická paradigmaty strojového učení): *učení s učitelem* (agent dostává správné odpovědi — např. učení modelu protivníka z pozorovaných tahů, klasifikace vjemů), *učení bez učitele* (hledání struktury v pozorováních — shlukování situací, objevování rolí) a **zpětnovazební učení** (k dispozici je jen skalární odměna z prostředí; pro autonomní agenty nejpřirozenější, viz zbytek kapitoly).
- **Imitační učení** (*learning from demonstration*): agent se učí politiku z trajektorií předvedených expertem — přímé kopírování (*behavioral cloning*), nebo *inverzní RL*, kde se z chování experta rekonstruuje odměnová funkce.
- **Evoluční metody**: populace kontrolérů (parametrů reaktivních chování, vah neuronové sítě, pravidel) se šlechtí genetickým algoritmem podle dosažené utility — typický způsob, jak se „experimentálně ladí“ reaktivní architektury z kap. 2; patří sem i *learning classifier systems* a neuroevoluce.
- **Podle toho, co se agent učí**: model prostředí (přechodovou funkci), model protivníka (*opponent modelling*), hodnotovou funkci, přímo politiku, u PRS ohodnocení plánů.
- **Individuální vs. sociální učení**: agent se učí jen z vlastní zkušenosti, nebo i napodobováním a sdílením zkušeností s ostatními (učení v roji, sdílené zkušenosti v MARL).

Nejběžnější a teoreticky nejpropracovanější přístupem k učení v MAS je **zpětnovazební učení** (*reinforcement learning*, RL) — změna chování metodou pokus–omyl na základě *odměn* z prostředí. RL zapadá do formalismu z kapitoly 1 přirozeně: politika π je mechanismus výběru akce a odměna je operacionalizací utility.

Rozlišujeme:

- **Učení jednoho agenta** — snazší, lze použít klasické algoritmy RL (Q-learning).
- **Multiagentní RL (MARL)** — agenti se učí *současně*; formalizuje se Markovskými hrami (Nashova a Paretova optimalita), state of the art je hluboké MARL (MADRL).

6.2 Markovský rozhodovací proces

MDP — Markov Decision Process

MDP je pětice $\langle S, A, T, R, \gamma \rangle$:

- S — prostor stavů; A — prostor akcí;
- $T : S \times A \times S \rightarrow [0, 1]$ — přechodová funkce, $T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$;
- $R : S \times A \times S \rightarrow \mathbb{R}$ — funkce odměny (okamžitá odměna za přechod);
- $0 \leq \gamma < 1$ — diskontní faktor, kompromis mezi okamžitou a budoucí odměnou.

Platí **Markovská vlastnost**: následující stav a odměna závisí pouze na aktuálním stavu a akci, nikoli na celé historii. (Srovnej s přechodovou funkcí $\tau : R^A \rightarrow 2^E$ z abstraktní architektury, která historii má — MDP je její markovské zjednodušení obohacené o pravděpodobnosti a odměny.)

Politika (*policy*) $\pi : S \rightarrow A$ (nebo stochasticky $\pi(a|s)$). Cílem je najít *optimální politiku* π^* maximalizující očekávaný diskontovaný součet budoucích odměn:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right].$$

Hodnotová funkce $V^\pi : S \rightarrow \mathbb{R}$ přiřazuje stavu očekávanou utilitu, sleduje-li agent politiku π ; **akční hodnotová funkce** $Q^\pi(s, a)$ totéž pro dvojici stav–akce.

Bellmanovy rovnice

Pro danou politiku (*Bellmanova rovnice pro evaluaci*):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')].$$

Pro optimální politiku (*Bellmanova rovnice optimality*):

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')],$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')].$$

Optimální politika se z Q^* získá jako $\pi^*(s) = \arg \max_a Q^*(s, a)$.

Iterace hodnot (*value iteration*) — známe-li úplný popis MDP:

- 1: inicializuj $V_0(s) \leftarrow 0$ pro všechna s
- 2: **repeat**
- 3: **for all** $s \in S$ **do**
- 4: $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$
- 5: **end for**
- 6: **until** $\max_s |V_{k+1}(s) - V_k(s)| < \varepsilon$
- 7: **return** $\pi(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$

Bellmanův operátor je kontrakce s koeficientem γ , takže iterace konverguje ke V^* geometricky rychle. Alternativou je **iterace politiky** (*policy iteration*): střídavě *vyhodnot* aktuální politiku (vyřeš soustavu lineárních rovnic) a *zlepší* ji (hladově podle V^π); konverguje v konečně mnoha krocích, protože politik je konečně mnoho.

6.3 Q-learning a SARSA

Ve skutečnosti obvykle **neznáme** T ani R . Agent se proto učí ze zkušenosti interakcí s prostředím v diskrétních krocích — to je *model-free* RL.

Q-learning (Watkins, 1989)

Agent si udržuje *tabulku* $Q(s, a)$ — odhad diskontovaného součtu budoucích odměn při provedení akce a ve stavu s . Po každém přechodu $s \xrightarrow{a} s'$ s odměnou r :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{cíl (TD target)}} - Q(s, a) \right], \quad 0 \leq \alpha \leq 1 \text{ je rychlost učení,}$$

ekvivalentně $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$.

Off-policy: cíl používá $\max_{a'}$, tedy se učí hodnotu *optimální* politiky bez ohledu na to, jakou politikou agent skutečně jedná. Za předpokladu, že každá dvojice (s, a) je navštívena nekonečněkrát a α vhodně klesá, konverguje $Q \rightarrow Q^*$.

Dilema průzkumu a využití; ε -greedy

Agent musí *využívat* (*exploit*) to, co už umí, ale zároveň *zkoušet* (*explore*) nové akce. Standardní řešení:

$$\pi(s) = \begin{cases} \text{náhodná akce z } A & \text{s pravděpodobností } \varepsilon, \\ \arg \max_a Q(s, a) & \text{s pravděpodobností } 1 - \varepsilon. \end{cases}$$

ε se typicky v čase snižuje (*annealing*). Alternativy: softmax/Boltzmann $\pi(a|s) \propto e^{Q(s,a)/\tau}$, optimistická inicializace Q , UCB (horní meze spolehlivosti).

Konkrétní krok Q-learningu. Nechť $\alpha = 0,5$, $\gamma = 0,9$. Agent je ve stavu s_1 , provede akci a_1 , dostane odměnu $r = 10$ a přejde do s_2 . Aktuální hodnoty: $Q(s_1, a_1) = 4$, $Q(s_2, a_1) = 6$, $Q(s_2, a_2) = 8$.

$$\max_{a'} Q(s_2, a') = \max(6, 8) = 8,$$

$$\text{TD cíl} = r + \gamma \max_{a'} Q(s_2, a') = 10 + 0,9 \cdot 8 = 17,2,$$

$$\text{TD chyba} = 17,2 - Q(s_1, a_1) = 17,2 - 4 = 13,2,$$

$$Q(s_1, a_1) \leftarrow 4 + 0,5 \cdot 13,2 = \mathbf{10,6}.$$

SARSA

State–Action–Reward–State–Action. Aktualizace používá *skutečně zvolenou* následující akci a' (podle stejné politiky, kterou agent jedná):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

On-policy: učí se hodnotu politiky, kterou agent skutečně provádí (včetně náhodného průzkumu). Důsledek: SARSA je *opatrnější* — v úloze „chůze po útesu“ (*cliff walking*) se Q-learning naučí optimální cestu těsně u propasti (a při ε -greedy do ní občas spadne), zatímco SARSA se naučí bezpečnější cestu dál od okraje, protože započítá riziko vlastního průzkumu.

Zobecnění obou: TD(λ) s *eligibility traces*, které rozprostírají kredit na více předchozích kroků.

6.4 Metody založené na politice

Policy gradient a actor–critic

Namísto hodnotových funkcí lze optimalizovat *přímo politiku*. Politiku parametrizujeme vektorem z , tedy $\pi(a \mid s, z)$, a hledáme z^* gradientním výstupem po očekávaném výnosu. **REINFORCE** (Williams, 1992) odhaduje gradient z Monte Carlo simulací celých epizod:

$$z_{t+1} = z_t + \alpha G_t \nabla_z \ln \pi(A_t \mid S_t, z_t),$$

kde G_t je výnos od času t — intuitivně: zvyš pravděpodobnost akcí, po nichž následoval vysoký výnos. Výhody oproti hodnotovým metodám: zvládá *spojité* akční prostory a přirozeně vytváří *stochastické* politiky (nutné v hrách se smíšenými ekvilibrii); nevýhodou je vysoký rozptyl odhadu gradientu.

Actor–critic kombinuje oba přístupy: *actor* reprezentuje politiku, *critic* se učí hodnotovou funkci a snižuje rozptyl. S **funkcí výhody** $A(s, a) = Q(s, a) - V(s)$ se gradient škáluje relativní výhodností akce místo výnosu. Varianty: A3C, A2C, PPO, SAC, DDPG.

Hluboké RL: DQN nahrazuje tabulku Q neuronovou sítí; klíčové triky jsou *experience replay* (rozbití korelací mezi vzorky) a *target network* (stabilizace cíle).

Proč hluboké RL potřebuje ty triky: smrtící trojice. Konvergenční záruky Q-learningu platí pro *tabulkovou* reprezentaci. Sejdou-li se současně tři věci — **aproximace hodnotové funkce** (neuronová síť místo tabulky), **bootstrapping** (cíl aktualizace obsahuje vlastní odhad $\max_{a'} Q(s', a')$) a **off-policy učení** (data pocházejí z jiné politiky, než kterou se učíme) — může učení *divergovat*. Sutton a Barto tuto trojici nazývají *deadly triad*. Každá dvojice sama o sobě je zvládnutelná; DQN se s ní vyrovnává právě replay bufferem a zamrzlou cílovou sítí, které oslabují druhý a třetí člen.

Tvarování odměny (*reward shaping*)

Řídká odměna (agent dostane signál až v cíli) činí učení prakticky neproveditelným, proto se odměna doplňuje o průběžné nápovědy $F(s, a, s')$. Naivní doplnění ale mění optimální politiku — agent se naučí sbírat nápovědy místo řešit úlohu.

Ng, Harada, Russell (1999): doplnění tvaru

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

pro libovolnou *potenciálovou funkci* $\Phi : S \rightarrow \mathbb{R}$ zachovává množinu optimálních politik — a je to jediný tvar, který to zaručuje pro všechny MDP. Intuice: součet takových přírůstků po libovolné cestě teleskopicky závisí jen na koncových bodech, nikoli na tom, kudy agent šel, takže cykly se nevyplatí.

6.5 Multiagentní zpětnovazební učení

Markovská hra (*Markov game*, stochastická hra)

Zobecnění MDP pro N agentů: $\langle N, S, (A_i)_{i \in N}, T, (R_i)_{i \in N}, \gamma \rangle$. **Sdružený akční prostor** je součin individuálních $\mathbf{A} = A_1 \times \dots \times A_N$; přechod závisí na akcích *všech* agentů a každý agent má *vlastní* odměnu R_i a politiku π_i , ale jeho hodnotová funkce závisí na politikách všech.

Spojení s teorií her: politika agenta je *nejlepší odpovědí* na sdružené politiky ostatních; sdružené politiky mohou být v **Nashově ekvilibriu** a mohou být **Pareto-optimální**. Speciální případy: $N = 1$ dává MDP; hra s jedním stavem dává hru v normální formě; $N = 2$ s nulovým součtem dává *minimax* hru.

Minimax-Q (Littman, 1994) — pro hry dvou agentů s nulovým součtem; místo $\max_{a'} Q(s', a')$ se použije hodnota smíšeného minimaxového ekvilibria matice $Q(s', \cdot, \cdot)$, což je lineární program. **Nash-Q** totéž pro obecný součet, ale konverguje jen za silných předpokladů.

Proč je MARL těžké

- Agenti aktualizují politiky **současně**, takže je prostředí z pohledu jednoho agenta **nestacionární** — porušuje se základní předpoklad konvergence RL („pohyblivý cíl“).
- Sdružený akční prostor roste **exponenciálně** s počtem agentů.
- Pro prostředí je jen **částečně pozorovatelné** (Dec-POMDP, optimální řešení je NEXP-úplné).
- **Přiřazení zásluh:** při společné odměně není jasné, který agent přispěl.

Standardní protiopatření: *centralizované učení s decentralizovaným prováděním* (CTDE) — kritik během tréninku vidí stav i akce všech agentů, herec při běhu jen svá lokální pozorování (MADDPG, COMA, QMIX); dále sdílení parametrů mezi homogenními agenty a *opponent modelling*.

Self-play. Agent se učí hraním proti sobě samému (případně proti dřívějším verzím sebe sama). V hrách s nulovým součtem a úplnou informací vede spolehlivě k silným strategiím: TD-Gammon, AlphaGo Zero, AlphaZero. Riziko jsou *cyklické* preference strategií (kámen–nůžky–papír), kdy naivní self-play osciluje — proto ligy protivníků a populační metody.

6.6 Shrnutí ke zkoušce

- Metody učení agentů: s učitelem / bez učitele / zpětnovazební; imitační učení a inverzní RL; evoluční metody (LCS, neuroevoluce); co se agent učí (model prostředí, model protivníka, politiku); individuální vs. sociální učení. RL je nejběžnější.
- **MDP** = $\langle S, A, T, R, \gamma \rangle$, Markovská vlastnost, politika π , hodnotové funkce V^π, Q^π ; Bellmanova rovnice optimality; value iteration (kontrakce, konverguje) a policy iteration.
- **Q-learning:** $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$, off-policy, tabulkový; umět spočítat jeden krok s čísly.
- **SARSA:** $\dots + \alpha[r + \gamma Q(s', a') - Q(s, a)]$, on-policy, opatrnější (cliff walking). ϵ -greedy jako řešení dilematu průzkum/využití.
- **Policy gradient** (REINFORCE, $\nabla \ln \pi$ krát výnos) a **actor-critic** (herec = politika, kritik = hodnotová funkce, výhoda $A = Q - V$); A3C jako asynchronní varianta; DQN (replay buffer, target network).
- **Smrtící trojice:** aproximace + bootstrapping + off-policy může divergovat — proto ty triky v DQN. **Tvarování odměny:** jen potenciálový tvar $F = \gamma \Phi(s') - \Phi(s)$ zaručeně nemění optimální politiku.
- **Markovská hra** = MDP pro N agentů se sdruženým akčním prostorem a individuálními odměnami; nejlepší odpověď, Nashovo ekvilibrium, Pareto-optimalita. **Minimax-Q** (nulový součet, LP v každé aktualizaci), Nash-Q (obecný součet).
- Problémy MARL: nestacionarita, exponenciální sdružený akční prostor, částečná pozorovatelnost, přiřazení zásluh; řešení CTDE. Self-play a AlphaZero.

7 Metodologie návrhu, jazyky a prostředí MAS

7.1 Agentově orientované softwarové inženýrství

Objektově orientované metodologie (UML) nestačí, protože agenti jsou autonomní (nelze přímo volat jejich metody), mají vlastní cíle a vztahy mezi nimi jsou *organizační* (role, protokoly, závazky), nikoli strukturální. Vznikly proto metodologie **AOSE** (*agent-oriented software engineering*).

Gaia (Wooldridge, Jennings, Kinny, 2000)

Jedna z nejcitovanějších AOSE metodologií. Klíčová metafora: MAS je **organizace**, agenti hrají **role**. Postupuje od požadavků přes analýzu k návrhu (implementaci nechává na konkrétní platformě).

Fáze analýzy — dva abstraktní modely. *Model rolí* (*roles model*): každá role je popsána čtyřmi atributy — *permissions* (jaké zdroje smí číst/měnit), *responsibilities* (co má zajistit; dělí se na *liveness* vlastnosti „něco dobrého se stane“, popsané regulárním výrazem nad aktivitami a protokoly, a *safety* vlastnosti „nic špatného se nestane“, popsané invarianty), *activities* (výpočty prováděné rolí samotnou) a *protocols* (interakce s jinými rolemi). *Model interakcí*: definice protokolů mezi rolemi (účel, iniciátor, respondér, vstupy, výstupy).

Fáze návrhu — tři konkrétní modely: *model agentů* (přiřazení rolí typům agentů a počty instancí), *model služeb* (funkce poskytované rolí — vstupy, výstupy, předpoklady, efekty) a *model znalostí* (*acquaintance model*: orientovaný graf, kdo s kým komunikuje — kontrola úzkých hrdel a přílišné provázanosti).

Omezení: původní Gaia předpokládá uzavřený systém s kooperativními agenty a statickou organizací; rozšíření *ROADMAP* a *Gaia v.2* tato omezení uvolňují.

♦ nad rámec slidů: slidy jmenují jen *Gaiu* a *role*; ostatní metodologie jsou orientační] **Prometheus** (Padgham, Winikoff, 2002) je praktická metodologie zaměřená na **BDI agenty** se třemi fázemi: *specifikace systému* (cíle, *funkcionalita*, rozhraní s okolím — *percepts* a *actions*), *architektonický návrh* (typy agentů, protokoly, zprávy) a *detaillní návrh* (*capabilities*, plány, události — přímo stavební kameny BDI). **Tropos** staví na modelování *aktérů a cílů* (framework *i**) a jako jediný pokrývá i fázi raných požadavků; pracuje s pojmy *actor*, *goal* a *softgoal* (nefunkční požadavek bez ostrého kritéria), *plan*, *resource* a *dependency*. Dále existují **MaSE**, **INGENIAS**, **ADELFE** a **PASSI**.

7.2 Standard FIPA a architektura platformy

Referenční architektura agentní platformy podle FIPA

- **AMS** (*Agent Management System*) — povinná komponenta; „matrika“ platformy: eviduje agenty, přiděluje jim jednoznačné *AID* (agent identifier), řídí životní cyklus (create, suspend, resume, terminate) a poskytuje bílé stránky (*white pages*).
- **DF** (*Directory Facilitator*) — žluté stránky (*yellow pages*): agenti u něj registrují *služby*, které nabízejí, a vyhledávají poskytovatele služeb.
- **MTS** (*Message Transport Service*) — doručování ACL zpráv v rámci platformy i mezi platformami (protokoly IIOP, HTTP; kódování bitefficient, XML, string).
- **ACC** (*Agent Communication Channel*) — rozhraní k MTS.

Standard dále definuje jazyk ACL, obsahové jazyky (SL), interakční protokoly a správu ontologií.

7.3 Jazyky a prostředí

♦ nad rámec slidů: AGENT-0 a Concurrent MetateM ve slidech nejsou; JADE a Jason ano] Historickým východiskem je **agentově orientované programování** (Shoham 1993) — paradigma, v němž je stav programu tvořen *mentálními kategoriemi* (přesvědčení, závazky, schopnosti) a program je množinou *pravidel závazků*. Prvním jazykem byl **AGENT-0**; jinou cestou jde **Concurrent MetateM**, kde je agent *přímo spustitelná specifikace* v temporální logice. Oba navazují na linii deduktivních agentů (odst.

2.1); prakticky nejúspěšnějším agentním jazykem je dnes AgentSpeak/Jason.

Systém	Typ	Charakteristika
JADE	Java, FIPA platforma	<i>Java Agent DEvelopment Framework</i> , jedna z nejrozšířenějších platform pro MAS, plně v souladu s FIPA. Agent = třída dědicí z Agent s metodou <code>setup()</code> ; chování se skládá z objektů Behaviour (OneShot , Cyclic , Ticker , FSMBehaviour , SequentialBehaviour), které plánuje kooperativní (nepreemptivní) plánovač uvnitř jednoho vlákna agenta. Zprávy: ACLMessage , <code>send()/blockingReceive()</code> , MessageTemplate . Hotové třídy pro protokoly (ContractNetInitiator , AchieveREResponder), ladicí nástroje (RMA, Sniffer, Introspector), kontejnery a mobilita agentů.
Jason	interpret AgentSpeak(L) v Javě	Referenční implementace AgentSpeak , tedy BDI jazyka (logické programování + BDI), přímý potomek PRS. Program agenta = počáteční <i>beliefs</i> , <i>rules</i> a <i>knihovna plánů</i> ve tvaru <code>spouštěč : kontext <- tělo</code> . Spouštěčem je <i>událost</i> (+b přidání přesvědčení, +!g přidání cíle, -!g selhání cíle). Cyklus interpretu: vnímej → aktualizuj beliefs → vyber událost → najdi <i>relevantní</i> plány (podle spouštěče) → vyber <i>aplikovatelné</i> (podle kontextu) → vytvoř záměr → vykoněj krok. S CARTAGO (prostředí) a Moise (organizace) tvoří <i>JaCaMo</i> .
NetLogo	prostředí pro ABM	Jazyk odvozený od Logo pro <i>agent-based modeling</i> . Entity: <i>turtles</i> (mobilní agenti), <i>patches</i> (buňky prostředí), <i>links</i> a <i>observer</i> . Knihovna modelů (Segregation, Wolf Sheep Predation, Ants, Flocking), <i>BehaviorSpace</i> pro parametrické studie. Určeno pro emergentní chování a výuku, ne pro FIPA komunikaci.
SPADE	Python, XMPP	<i>Smart Python Agent Development Environment</i> . Komunikace přes standardní XMPP místo vlastního MTS, agent = třída s <i>behaviours</i> (CyclicBehaviour , PeriodicBehaviour , FSMBehaviour), <i>asyncio</i> , webové rozhraní; populární díky napojení na Python ekosystém strojového učení.

Další prostředí: *Jadex*, *JACK* a *JAM* (další implementace BDI/PRS, Jadex staví BDI vrstvu nad JADE); *MASON*, *Repast* a *GAMA* (simulační platformy pro agentní modely); *PettingZoo*, *Gymnasium* a *OpenSpiel* (standardní prostředí pro multiagentní zpětnovazební učení).

7.4 Agenti postavení na velkých jazykových modelech

◆ nad rámec slidů: nově ve slidech verze 2025 (kapitola *LLM Agents*)

Poslední přírůstek do rodiny mechanismů výběru akcí (§1.9) nepoužívá ani dedukci, ani pravidla, ani naučenou *Q*-funkci, nýbrž **velký jazykový model** (LLM). Agent je pak smyčka, v níž se stav prostředí zakóduje do textového promptu a výstup modelu se interpretuje jako akce.

LLM v roli komponenty agenta

- **LLM jako politika** (*action selection*). Prompt obsahuje popis role, stav prostředí a seznam dostupných akcí; výstupem je zvolená akce. Odpovídá funkci $action : I \rightarrow A$ z abstraktní architektury — vnitřním stavem *I* je kontextové okno (případně externí paměť).
- **LLM jako plánovač**. Model dekomponuje cíl na podcíle a generuje kandidátní plány; prohledávání prostoru plánů řídí buď model sám (*chain of thought*, *ReAct*), nebo vnější prohledávací algoritmus, jemuž model slouží jako generátor nástupníků a jako heuristika.
- **LLM jako kritik**. Ve schématu *actor-critic* (§6.4) hodnotí model kvalitu provedené akce nebo celé trajektorie a poskytuje zpětnou vazbu tam, kde prostředí skalární odměnu nedává.
- **LLM jako rozhraní k nástrojům** (*tool use*). Model volí, který externí nástroj (vyhledávání, kalkulátor, API, jiný agent) zavolat a s jakými argumenty — to je přímá obdoba *služeb*

registrovaných v DF a volání přes ACL.

- **Učení s člověkem ve smyčce** (*human-in-the-loop*): člověk opravuje a doplňuje zpětnou vazbu, podle níž se upravuje prompt, paměť nebo váhy modelu.

Typické nasazení — agentové simulace. Nejčastější aplikací nejsou optimalizační úlohy, ale *agent-based modeling* (§7.3, NetLogo), kde LLM agent nahradí jednoduchého reaktivního agenta a dodá mu doménovou znalost i jazykové chování:

- *Šíření informací v sociální síti.* Agenti jsou uzly grafu sociálních interakcí, zprávu si předávají v přirozeném jazyce a sleduje se rychlost a tvar šíření zprávy sítí.
- *Syntetický volební panel.* Populace agentů je vygenerována podle sociologických průzkumů (demografie, postoje) a používá se k odhadu výsledků voleb nebo reakce na kampaň.

Zařazení a kritika. Z hlediska okruhu jde o další rodinu mechanismů výběru akcí — subsymbolickou jako neuronové sítě, ale s výrazně širší doménovou znalostí a bez nutnosti trénovat model úlohy. Slabiny jsou zrcadlovým obrazem jejích silných stránek: agent nemá explicitní model světa (nelze o něm dokazovat tvrzení jako u deduktivního agenta), sémantika jeho „přesvědčení“ je **neverifikovatelná** ve stejném smyslu jako u FIPA-ACL (§4.7), chování je nedeterministické a cena jednoho rozhodnutí je o několik řádů vyšší než u pravidlového agenta.

7.5 Shrnutí ke zkoušce

- Proč nestačí OO metodologie: autonomie, vlastní cíle, organizační (nikoli strukturální) vztahy.
- **Gaia**: organizace + role; analýza = model rolí (permissions, responsibilities s liveness a safety, activities, protocols) + model interakcí; návrh = model agentů, služeb a znalostí. Předpoklad kooperativních agentů a statické organizace.
- **Prometheus** (cíleno na BDI: funkcionality → typy agentů → capabilities a plány) a **Tropos** (aktéři, cíle a softgoals, závislosti; pokrývá i rané požadavky).
- **FIPA platforma**: AMS (bílé stránky, životní cyklus, AID), DF (žluté stránky, služby), MT-S/ACC (doručování ACL zpráv).
- **JADE** = Java, FIPA, Agent + Behaviour + ACLMessage, hotové protokoly a ladicí nástroje. **Jason/AgentSpeak** = BDI jazyk, plán spouštěč : kontext <- tělo, potomek PRS. **NetLogo** = ABM simulace (turtles/patches/links). **SPADE** = Python + XMPP.
- **LLM agenti**: jazykový model jako *politika* (prompt → akce), *plánovač*, *kritik* v actor-critic a rozhraní pro *volání nástrojů*; nasazení hlavně v agentových simulacích (šíření zpráv sítí, syntetický volební panel). Další rodina mechanismů výběru akcí — bez explicitního modelu světa a s neverifikovatelnou sémantikou přesvědčení, zato se širokou doménovou znalostí.

Závěrem: jak na zkoušku

Okruh je široký; zkoušející obvykle ověřuje, že rozumíte *souvislostem*. Čtyři osy, kolem kterých se otázky točí:

1. **Reaktivita vs. proaktivita** — od definice inteligentního agenta přes commitment strategie (bold vs. cautious) a parametry γ/ϕ v síti Maes až po vrstvy hybridních architektur.
2. **Individuální vs. kolektivní racionalita** — sobecký agent vede k (D, D) ve vězňově dilematu, k tragédii obecní pastviny i k taktickému hlasování; odtud potřeba mechanism designu (Vickrey, VCG), tedy návrhu *pravidel*, za nichž se sobeckému agentovi vyplatí být pravdomluvný.
3. **Symbolická vs. subsymbolická reprezentace** — FOL, STRIPS, ontologie a ACL versus subsumpce, aktivační sítě a neuronové sítě v RL; praxe je hybridní.
4. **Teorie vs. praxe** — elegantní, ale nekonstruktivní definice (optimální agent, Nashova věta, Arrowova věta) versus použitelné algoritmy (A^* , CBS, Q-learning, CNET, JADE). Nezapomeňte na složitost: PPAD-úplnost hledání NE, NP-těžkost WDP i optimálního MAPF.

Drobnosti, na které se často ptají: rozdíl mezi *desire* a *intention*; proč efektem **inform** *není*, že příjemce uvěří; proč je (D, D) jediné *ne*-Pareto-optimální řešení věžňova dilematu; proč je pravdomluvnost dominantní ve Vickreyově aukci; rozdíl mezi off-policy Q-learningem a on-policy SARSA; a co dělá CBS na vysoké a co na nízké úrovni.